

Wrocław University of Science and Technology
Faculty of Information and Communication Technology

Field of study: **Applied Computer Science (IST)**

Speciality: **Software Engineering (IO)**

MASTER THESIS

**Learning code change representations via
artificial intelligence code model**

Jakub Zehner

Supervisor

dr hab. inż. Lech Madeyski, prof. uczelni

Keywords: code change representation, code change, representation learning, code embedding,
pre-trained model

Abstract

Context: Code change representation (CCR) involves converting code changes from textual form to numerical feature vectors. *Objective:* This thesis aims to organize existing research, identify knowledge gaps, and propose a new CCR approach. *Method:* A systematic review (SR) was conducted. The influence of different pre-trained models on the performance of the state-of-the-art method, CCRep, was analyzed. A novel retrieval augmentation (RA) approach was proposed, previously unexplored in CCR, as an alternative to costly fine-tuning of large models. An experimental framework, RAFramework, was developed to support the evaluation. *Results:* The SR identified 95 relevant publications and highlighted the lack of general-purpose CCRs. The study showed that encoder choice has a limited impact on the effectiveness of CCRep. The proposed RA-based method performed comparably to CCRep on large datasets, while being less computationally demanding. *Conclusions:* The RA-based technique introduced in this work offers a scalable and flexible solution for general-purpose CCR.

Streszczenie

Kontekst: Reprezentacja zmian w kodzie (RZK) polega na przekształceniu zmian w kodzie z postaci tekstowej na numeryczne wektory cech. *Cele:* Celem niniejszej pracy jest uporządkowanie istniejącego dorobku badawczego, identyfikacja luk w wiedzy oraz zaproponowanie nowego podejścia do RZK. *Metoda:* Przeprowadzono systematyczny przegląd literatury (SPL). Zbadano wpływ zastosowania różnych wstępnie wytrenowanych modeli na skuteczność rozwiązania CCRep, uznawanego za stan obecnej wiedzy. Ponadto zaproponowano nowatorskie, niestosowane dotąd w RZK, podejście oparte na wzbogacaniu przez wyszukiwanie (WPW), jako alternatywę dla kosztownego dostrajania dużych modeli. W celu przeprowadzenia eksperymentów opracowano dedykowany framework badawczy – RAFramework. *Wyniki:* SPL pozwolił zidentyfikować 95 istotnych publikacji i wskazało brak metod RZK ogólnego zastosowania. Badania empiryczne wykazały, że wybór enkodera ma ograniczony wpływ na skuteczność CCRep. Zaproponowana metoda, WPW, osiągnęła porównywalne wyniki do CCRep na dużych zbiorach danych, będąc mniej wymagającym obliczeniowo. *Wnioski:* Przedstawione w pracy podejście oparte na WPW stanowi obiecujące, skalowalne i elastyczne rozwiązanie dla ogólnych zastosowań RZK.

Contents

Introduction	3
Background	3
The aim of thesis	4
The scope of thesis	5
1. Related work	6
1.1. Systematic literature review	6
1.1.1. Protocol of the systematic literature review	7
1.1.2. Description of the systematic literature review method	7
1.1.3. Course of the systematic literature review	8
1.1.4. Result of the systematic literature review	11
1.2. Taxonomy of code representation techniques	17
1.2.1. Lexical code representation	17
1.2.2. Tree-based code representation	18
1.2.3. Graph-based code representation	20
2. Proposed solution and its implementation	23
2.1. System infrastructure	23
2.1.1. Query back experimentation module	23
2.1.2. Retrieval augmentation experimentation framework	24
2.2. Use of pre-trained models	25
2.3. Approach based on the query-back mechanism	28
2.4. Approach based on retrieval augmentation	29
2.4.1. Methods of retrieval embeddings	31
2.4.2. Methods of combining embeddings	32
3. Empirical evaluation of the proposed solution	35
3.1. Downstream tasks	35
3.2. Overview of selected datasets	36
3.3. Evaluation metrics	37
3.3.1. Evaluation metrics for binary classification tasks	37
3.3.2. Evaluation metrics for text generation task	39
3.4. Research questions	40
3.5. Results and analysis	42
3.5.1. Impact of various encoders on query back mechanism	42
3.5.2. Performance of the proposed retrieval augmentation approach	45

3.5.3. Key findings from the empirical evaluation	50
4. Discussion and further work	51
4.1. Variants of the proposed approach	51
4.2. Limitations	51
4.3. Threats to validity	52
4.3.1. Validity of the systematic literature review	52
4.3.2. Validity of the Solution and Its Evaluation	53
4.4. Further work	54
Conclusion	55
Acknowledgements	57
Bibliography	58
List of Figures	67
List of Listings	68
List of Tables	69
Appendix	70
A. Systematic review protocol	71
Research questions	71
Search process	71
Inclusion criteria	72
Exclusion criteria	72
Data collection	72
Data analysis	73

Introduction

The Introduction outlines the research problem, motivations, and the existing gap in the current state of knowledge related to learning code change representations (CCR) via artificial intelligence code models. It highlights the importance of understanding and modeling software changes, especially in the context of automated software engineering, maintenance, and code evolution. The chapter also presents the main objectives and scope of the thesis, defining the specific challenges addressed in this work, and explaining how the proposed approach contributes to the advancement of intelligent code analysis techniques.

Background

The software development process is an integral part of a developer’s daily work. During this process, programmers introduce numerous changes to the source code. Each new feature, bug fix, removal of obsolete code, or refactoring operation results in a code change that becomes part of the project’s evolution history [1, 2]. This history can be viewed as an ordered sequence of code modifications [3], which can be represented in various ways, for example, by juxtaposing the code before and after a change, or as flat text, such as *diff* [4].

Understanding and analyzing these code changes is not only useful in documenting the history of a project’s development, but also provides a valuable foundation for automating various tasks in the field of software engineering (SE). Such tasks include, but are not limited to, automated patch correctness assessment (APCA) [5] and just-in-time defect prediction (JIT-DP) [6] – both of which require a detailed analysis of vulnerabilities or defects captured within a code change. These tasks involve reasoning about whether a change introduces or removes a bug. Another task, commit message generation (CMG) [7], focuses on summarizing the intent and content of a code change to generate a meaningful and human-readable message. All of these tasks require a deep semantic understanding of code modifications [8–10].

A crucial prerequisite for enabling such intelligent automation is the ability to extract a suitable and meaningful CCR. One of the widely adopted and effective strategies for this purpose is to encode changes as numerical feature vectors that can be further processed. Intuitively, according to the principle "garbage in, garbage out" [11], the quality of the CCR plays a critical role in the downstream performance of these tasks. High-quality

representations serve as informative and computationally convenient input that can enhance the accuracy and efficiency of target applications [2].

In traditional machine learning (ML) approaches, the construction of such feature vectors was heavily based on manual feature engineering, that is, the process of designing and extracting relevant information from code by human experts. This process was not only labor intensive and time consuming, it also required substantial domain expertise and lacked generalizability across different programming languages, technologies, and tasks. In response to these limitations, recent years have seen a paradigm shift toward *representation learning*, which involves automatically learning useful features from raw data without the need for hand-crafted feature extraction. In this context, deep learning (DL) neural networks have proven to be highly effective due to their ability to capture complex patterns in data [12]. More recently, the rise of the Transformer architecture [13] has revolutionized the field, establishing pre-training and fine-tuning as the dominant approach in SE tasks [14–17]. In this paradigm, large language models are first pre-trained on vast corpora of code using self-supervised learning, and subsequently fine-tuned for specific tasks. This approach has shown great promise in learning CCR, where pre-trained models (PTMs) are used to extract task-relevant information from code modifications. These PTMs often outperform traditional ML [7] and information retrieval (IR) [5] methods, achieving state-of-the-art results.

However, the vast majority of existing CCR approaches are task-specific, which means they are tailored to solve a particular problem and are not easily adaptable to other contexts, especially those with limited training data. There is a noticeable scarcity of general purpose CCR methods that can produce high-quality representations that can be used in a wide range of downstream tasks [12]. Moreover, some existing approaches suffer from critical limitations: for example, some can only handle very small code changes (e.g., up to three lines) [18], while others rely on the presence of commit messages to supervise the learning process [2]. This dependence on commit messages is problematic, as such messages are not always available [5, 19], and collecting well-formed, high-quality messages is itself a non-trivial challenge [7, 20, 21].

The aim of thesis

Given the limited availability of general-purpose solutions for learning CCR and their growing significance in the field of software engineering, this thesis focuses on the development of a universal and robust approach to learning code change representations via artificial intelligence code models. The proposed approach aims to be adaptable across various downstream tasks, independent of auxiliary inputs such as commit messages, and capable of capturing both the syntactic and semantic aspects of code changes.

In addition, the development of the new CCR model is preceded by a systematic review of the literature, which serves to summarize the current state of knowledge in the field. This review aims to identify existing approaches, assess their strengths and limitations, and highlight gaps and challenges that remain unaddressed.

The scope of thesis

This thesis is organized into four main chapters, each addressing a specific aspect of the research. The first chapter outlines the methodology and findings of a systematic review of the literature, providing an overview of existing approaches and related work in the field of CCR. The second chapter introduces the proposed solution, including an explanation of its design and implementation. Chapter three presents the empirical evaluation of the method developed and analysis of the results. The fourth chapter offers a critical discussion of the findings, explores the limitations of current work, and suggests possible directions for future research. The thesis concludes with a summary.

1. Related work

The purpose of this chapter is to present the motivation, methodology and key findings of the systematic literature review conducted for this thesis. The review aims to provide a clear overview of the current state of research in the field of CCR, highlighting existing approaches, their applications, and limitations.

1.1. Systematic literature review

In order to summarize the current state-of-the-art in the field of CCR and identify existing approaches along with their strengths and weaknesses, a systematic literature review was conducted.

A systematic literature review, often referred to simply as a systematic review (SR), is a rigorous method for identifying, evaluating, and interpreting all available research relevant to a specific research question or area of study. In contrast to traditional or narrative literature reviews, which are often subjective and unsystematic, SRs follow a formalized methodology with clearly defined objectives and explicit steps [22]. This structured approach is particularly valuable in software engineering, where a large amount of research requires careful filtering and analysis.

For this thesis, the snowball technique proposed by Wohlin [23] was adopted as the primary method for the SR. Snowballing is a search strategy that begins with a set of relevant primary studies, referred to as the "start set". From these, backward snowballing involves examining the references cited in the start set, while forward snowballing involves identifying subsequent publications that cite the start set studies. This iterative process continues until no new relevant papers are found, thus expanding the coverage of the literature beyond what might be retrieved through conventional keyword-based searches.

Furthermore, the review process followed the general guidelines for the conducting of SRs in software engineering as described in 2007 by Kitchenham et al. [22] and further elaborated by SEGRESS [24] in 2023. However, it is important to note that these guidelines were applied with some simplifications. This decision was motivated by two main factors: first, the guidelines are primarily intended for larger research teams rather than individual researchers; second, within the context of a master's thesis, the SR constitutes only one component of the overall work, rather than being its central focus. Therefore, a pragmatic approach was taken to balance methodological rigor with the scope and practical constraints of the thesis.

1.1.1. Protocol of the systematic literature review

According to the recommendations outlined by Kitchenham et al. [22], the SR was preceded by a planning phase during which a detailed review protocol was developed. This protocol is included in Appendix A.

The purpose of the review protocol is to formally define the scope, methodology, and objectives of the SR, ensuring that the process remains consistent, transparent, and reproducible. It also serves as a guide for conducting the SR in a rigorous and structured manner.

The protocol outlines the motivations for conducting the review and defines the following systematic review research questions (SRRQ), which serve as the foundation for data collection and analysis.

- **SRRQ1:** *What are the state-of-the-art general-purpose models for representing code and code changes?*
- **SRRQ2:** *What are the state-of-the-art models tailored for specific tasks, such as defect prediction or commit message generation, in the context of code and code change representation?*
- **SRRQ3:** *What data sets are employed in the learning of representations for code changes?*

In addition to the SRRQs, the protocol specifies the intended methodology for conducting the review, including the selection of the search strategy, the criteria for including and excluding papers, and the approach to data collection and analysis. All of these components are crucial for maintaining the integrity of the review process.

The protocol also describes how the collected data will be processed and analyzed. Specifically, it outlines how relevant metadata and qualitative observations from each article will be recorded and how significant findings will be synthesized to address research questions.

Although the protocol is based on the comprehensive template provided in Kitchenham et al. [22], it has been appropriately simplified to reflect the scope of a master's thesis. The complete protocol is provided in Appendix A and documents all decisions made during the planning and execution of the review process.

1.1.2. Description of the systematic literature review method

The SR in this thesis was carried out using the snowballing methodology, as described by Wohlin [23]. Snowballing is an iterative search technique designed to identify relevant literature by exploring the citation network surrounding an initial set of papers, called the *start set*.

The method involves two main phases:

- **Backward snowballing** – This phase entails reviewing the reference lists of the papers included in the current iteration to identify earlier works that are potentially relevant to the research questions.
- **Forward snowballing** – In this phase, the focus is on identifying subsequent publications that cite the papers already included in the review. Wohlin [23] recommends the use of tools such as Google Scholar to carry out this step effectively.

Each iteration of the snowballing process results in a new set of non-duplicate papers that meet the inclusion criteria and contribute meaningfully to the systematic review. These papers are then added to the review set, and the process is iteratively continued. The cycle is repeated until no new significant papers are identified, indicating that theoretical saturation has been reached.

The general snowballing procedure used in this thesis is illustrated in Figure 1.1, which visually represents the forward and backward citation expansion process in multiple iterations.

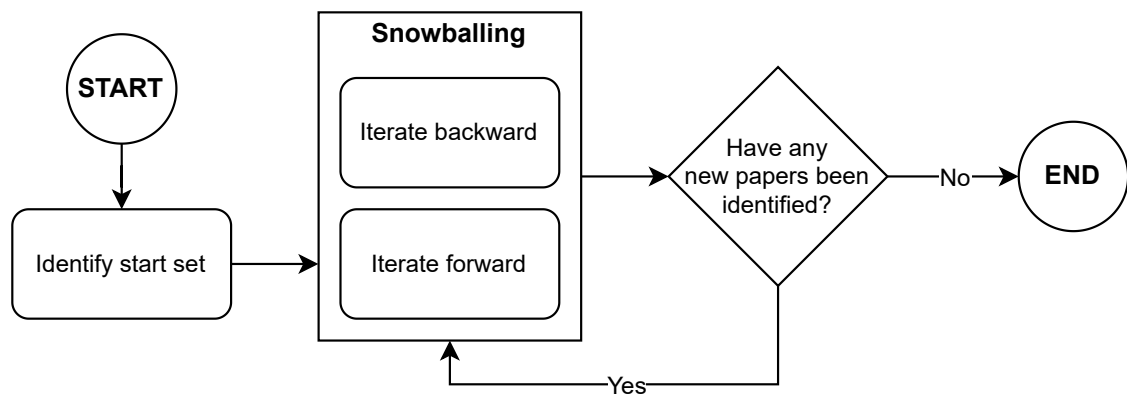


Fig. 1.1. Steps of the snowballing procedure.
Own work, based on [23].

Although not an obligatory component of the snowballing methodology, Wohlin [23] suggests an additional step at the end of the process: contacting the authors of the papers found, especially the most active ones. The aim of this step is to discover relevant studies that might have been missed through the citation network. If such studies are identified, the snowballing process should be resumed to incorporate them.

However, due to the simplified scope of this master's thesis and time constraints, this optional final step was omitted. This decision is consistent with other adaptations made to reduce the review process for a single-author academic project, as discussed in Section 1.1.

1.1.3. Course of the systematic literature review

SR was carried out according to the review protocol previously established and documented in Appendix A, and discussed in Section 1.1. The general procedure is also

visualized in Figure 1.2, which illustrates the steps of the snowball-based review process adapted from Wohlin [23].

The review began with the analysis of two papers provided by the thesis supervisor, namely: CCRep [3] and TransformCode [25]. These papers served as a foundational set (*initial papers*) from which key concepts, terminology, and index terms were extracted. Table 1.1 summarizes the keywords identified in both publications.

Table 1.1. Index terms in initial papers from supervisor.

Paper	Index terms
CCRep Liu et al. [3]	<i>code change, representation learning, commit message generation, patch correctness assessment, just-in-time defect prediction</i>
TransformCode Xian et al. [25]	<i>code embedding, transformer, abstract syntax tree, contrastive learning</i>

Based on these key terms, a search query was formulated and applied to the Scopus database with the objective of assembling the *start set* for the snowballing process. The complete query is presented in the Listing 1.1. The construction of the query was guided by the inclusion and exclusion criteria defined in the review protocol. These criteria were defined as follows:

Inclusion Criteria:

- The study addresses the topic of CCR.
- The study qualifies as a systematic review of the literature or contributes substantially to one or more of the stated research questions.
- The study offers empirical or conceptual insights relevant to CCR.

Exclusion Criteria:

- The paper is not written in English.
- The paper lies outside the domain of Computer Science.
- The paper has not been peer reviewed or formally published.
- The paper is inaccessible due to paywall restrictions and cannot be obtained through institutional or open access repositories.

An additional constraint was applied to limit the publication year to 2019 and later. This decision was based on the emergence of a landmark publication in 2019 – Code2Vec by Alon et al. [26], which has garnered over 1500 citations and is widely regarded as a foundational work in modern code representation learning. By including only papers published after this milestone, the query targeted recent and relevant advancements in the field.

The query returned a total of 226 results (as of January 31, 2025), including both of the *initial papers*, indicating that the query was sufficiently broad and not overly restrictive. The complete Scopus search query used is provided in the following Listing 1.1.

```
( ( TITLE-ABS-KEY("representation learning")
  AND TITLE-ABS-KEY("code change") )
  OR ( TITLE-ABS-KEY("code embedding") )
  OR ( TITLE-ABS-KEY("code change representation") ) )
AND ( LIMIT-TO(PUBSTAGE , "final") )
AND ( LIMIT-TO(SUBJAREA , "COMP") )
AND ( LIMIT-TO(LANGUAGE , "English") )
AND PUBYEAR > 2018
```

Listing 1.1: The Scopus query used to gather papers for the start set.

Following the execution of the Scopus search query, the resulting list of publications was manually reviewed in accordance with the predefined inclusion and exclusion criteria. This manual selection process led to the selection of 51 additional papers. Together with the two supervisor-provided papers discussed earlier, these form the final *start set* for the snowballing procedure.

The constructed *start set* aligns well with the guidelines proposed by Wohlin [23], ensuring a diverse representation of the works in different authors, publications years, publishers, and research communities. This diversity is essential to reduce the risk of bias and maximize the coverage of relevant literature. However, it is important to acknowledge a potential limitation in this step. Since the initial screening was conducted by a single reviewer (the author of this thesis), there is a possibility of subjectivity or selection bias. However, this risk was mitigated in the subsequent snowballing stages, which systematically broadened the scope of the review in a replicable and transparent manner.

During this phase, several key publications were identified, judged by their citation count and perceived influence within the research community. Among them, Code2Vec by Alon et al. [26] stands out as a seminal contribution to the field of code representation learning. In addition, important works such as FIRA proposed by Dong et al. [27], Commit2Vec by Cabrera Lozoya et al. [28], and models introduced by Cheng et al. [29] and Lin et al. [19] were also included. These papers present various methodological approaches and are applied to various downstream software engineering tasks such as automated commit message generation, vulnerability detection, and defect prediction.

The *start set*, comprising 53 papers in total, contained 2,642 references and had collectively received 2,180 citations at the time of analysis. These reference and citation networks formed the basis for conducting the first iteration of the snowballing procedure.

During the backward snowballing phase, the reference lists of all papers in the *start set* were examined. This was initially done by screening the titles of the referenced papers. In

cases where the title was ambiguous or insufficient to determine relevance, the abstracts were reviewed. If further clarification was needed, a quick inspection of the full text was conducted.

For the forward snowballing phase, Google Scholar was used to identify publications that cited works from the *start set*. As with backward snowballing, the titles and abstracts of the citing papers were evaluated, with additional inspection of the full content where necessary. In cases where a paper had a very high number of citations, a full inspection of all cited papers was not feasible. However, at least the first 10 pages of the results, sorted by relevance by default, were always reviewed to ensure the inclusion of the most pertinent and influential studies.

The snowballing process was carried out iteratively. In each iteration, newly identified relevant papers were added to the pool and their references and citations were used in subsequent iterations. The process was concluded after four iterations, as no additional relevant and unique papers were identified in the final round.

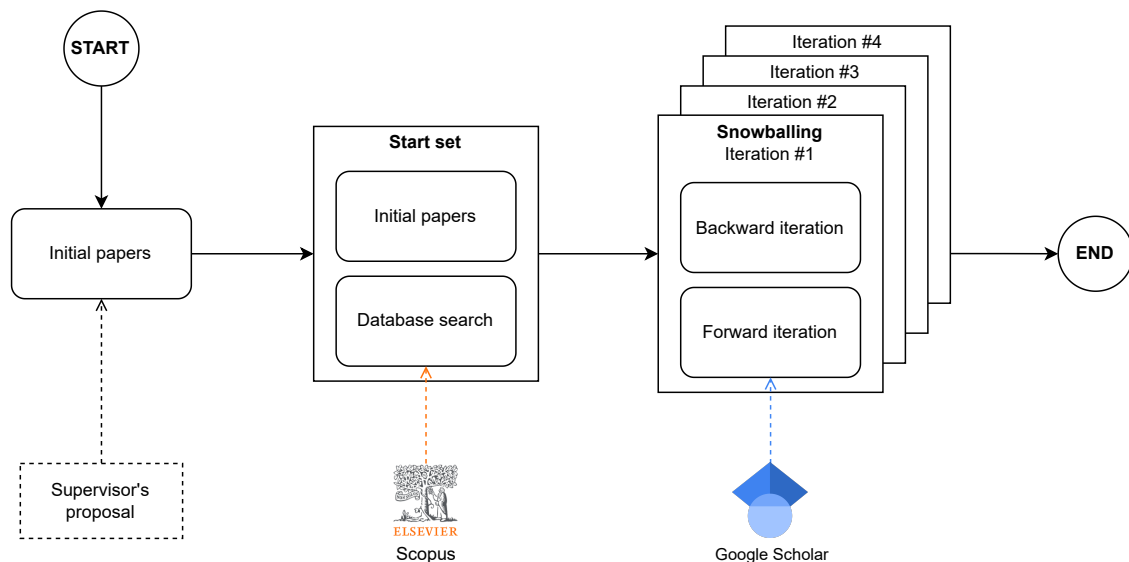


Fig. 1.2. The course of the literature review.
Own work.

Detailed statistics on the number of papers identified in each iteration and other relevant data are presented in the following Section 1.1.4, which discusses the general results of the systematic review.

1.1.4. Result of the systematic literature review

As a result of the SR, a total of 95 unique papers relevant to the predefined research questions were identified. This final set was distilled from a pool of nearly 18,000 publications reviewed throughout the various stages of the process (including duplicates). A detailed summary of the number of papers identified and reviewed in each phase is presented in Table 1.2.

Table 1.2. The number of papers identified and reviewed at each stage of the literature review.

Review stage		Identified papers (new & unique)	Reviewed papers (including duplicates)
Start set	Initial papers	2	—
	Database search	51	226
Iteration #1	Backward snowballing	29	2642
	Forward snowballing	4	2180
Iteration #2	Backward snowballing	1	1868
	Forward snowballing	6	10014
Iteration #3	Backward snowballing	0	478
	Forward snowballing	2	223
Iteration #4	Backward snowballing	0	109
	Forward snowballing	0	70

Figure 1.3 illustrates the distribution of the identified papers by publication year. As shown in the chart, significant advancements in the area of learning code representations began emerging only in recent years. Before 2019, the topic was relatively underexplored. A turning point in this research area was marked by the publication of Code2Vec by Alon et al. [26], which introduced an influential learning representation model for code. This aligns with the observation that only two relevant works were identified before 2019.

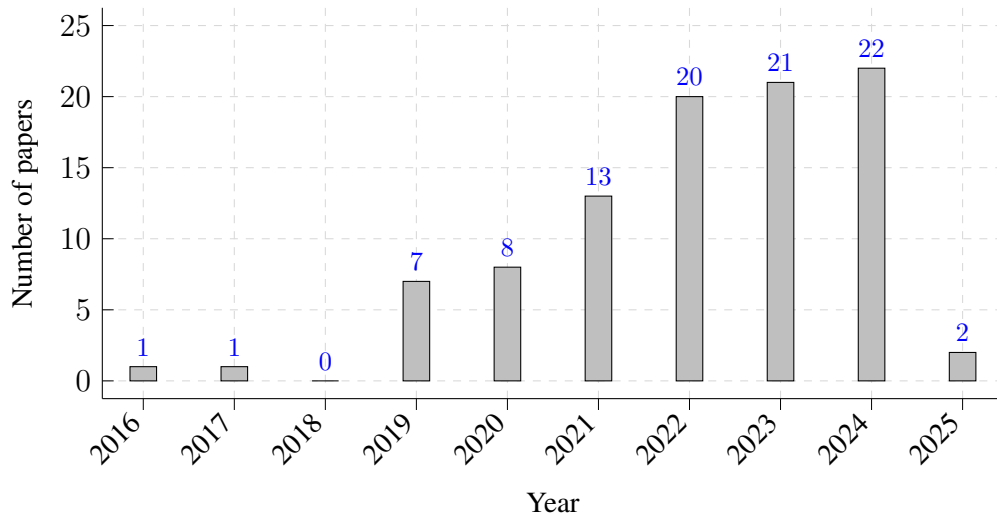


Fig. 1.3. Distribution of identified papers by publication year.

The first paper identified in the review is from 2016, authored by White et al. [30], which applied deep learning techniques to clone detection, an early attempt at learning code

representations. Another notable contribution is the 2017 study by Li et al. [31], which explored representation learning for defect prediction. These early studies were among the first to adopt machine learning approaches that do not rely on handcrafted features, marking a clear departure from the dominant methodologies of the time.

Since 2019, a noticeable and consistent growth in the number of publications related to code representation learning has been observed. This upward trend reflects an increasing academic interest and a shift away from traditional machine learning models toward data-driven approaches that leverage learned representations.

Among the reviewed papers, another clear trend emerged: the increasing use of Transformer-based architectures, initially introduced in the groundbreaking work *Attention Is All You Need* by Vaswani et al. [13]. These architectures have rapidly become the foundation for many state-of-the-art models in software engineering tasks, particularly in the context of code understanding and generation. Moreover, a growing number of works now rely on large PTMs, which leverage massive corpora of source code to learn general-purpose representations.

The systematic review also identified two prior systematic reviews that are tangentially related to the topic of CCR. The first, conducted by Shiri Harzevili et al. [32], focuses on automated software vulnerability detection using machine learning. Although it does not directly address CCR, one of its research questions pertains to input representations and code embedding methods, which makes it partially relevant. The second review, by Samoaa et al. [12], more directly addresses the issue of source code representation, providing a broader overview of encoding strategies and abstraction techniques used in learning from source code.

SRRQ1: *What are the state-of-the-art general-purpose models for representing code and code changes?*

The data collected during the SR was used to address the SRRQ1. Based on the analysis of the selected literature and to the best of the author’s knowledge, the CCRRep model, proposed by Liu et al. [3], currently represents the state of the art in the domain of CCR. The model introduces a novel mechanism, called query back, which extracts and encodes modified code segments in a way that ensures that they interact effectively with the entire code-change context. This mechanism enhances the ability of the model to focus on the most relevant parts of a change, thus improving the quality of the resulting representations.

CCAF has achieved a comparable level of performance, proposed by Zou et al. [33], which adapts the AdapterFusion technique introduced by Pfeiffer et al. [34] to improve knowledge fusion in multiple tasks. Notably, both CCRRep and CCAF utilize PTMs and are model-agnostic, meaning that they can be integrated with various base encoders without being tightly coupled to a specific architecture.

The next method focusing on CCR, the Commit2Vec model by Cabrera Lozoya et al. [28] introduces a representation learning technique that leverages abstract syntax trees (ASTs) to construct embeddings of code changes, effectively capturing the syntactic structure of commits. Another approach is CC2Vec by Hoang et al. [2], which models the structural aspects of code changes and applies an attention mechanism to identify and emphasize the most critical components of a change. Although these models are relatively older, they laid the groundwork for subsequent research and development in general-purpose CCR models.

Building on the foundation established by CC2Vec, Zhang et al. [35] proposed a slice-based code change representation approach. This method incorporates both data and control dependencies into the representation process and introduces a corresponding PTM named CCS2Vec. By incorporating program slicing into the representation, this approach provides a more semantically rich understanding of code changes.

Another important PTM identified during the review is CCBERT, introduced by Zhou et al. [36]. CCBERT is trained on a large-scale dataset of real-world code changes and demonstrates improved performance over CodeBERT [14] in CCR-related tasks. In addition to its superior accuracy, CCBERT is also more computationally efficient, requiring less GPU memory and shorter training times.

Although SRRQ1 focuses primarily on CCR models, it is also relevant to consider general-purpose models that learn representations of source code. This broader category includes a significantly larger body of work. One such example is CoCoAST by Shi et al. [37], which decomposes the AST of a source code file into multiple subtrees, embeds each subtree individually, and aggregates these embeddings to form a complete representation of the original AST. This hierarchical approach allows the model to better capture both local and global structural information.

In addition, a wide array of PTMs for code representation have been proposed in recent years. These include: GraphCode2Vec [38], which leverages program graphs for learning representations; CodeT5 [16], a unified encoder-decoder model designed for a variety of code understanding and generation tasks; UniXcoder [39], a cross-modal model capable of processing both code and natural language; Graphcodebert [15], which integrates both the syntactic and semantic code structure into the representation; CodeBERT [14], a foundational model widely used in software engineering tasks; And also the previously mentioned Code2Vec model [26].

SRRQ2: *What are the state-of-the-art models tailored for specific tasks, such as defect prediction or commit message generation, in the context of code and code change representation?*

The results collected during the SR also allowed the formulation of an answer to SRRQ2. One of the most frequently explored downstream tasks, as identified in the

reviewed literature, is the CMG. Among the models designed for this task, a notable contribution is the method proposed by Dong et al. [27], which introduces a novel technique named FIRA. This approach represents code changes using fine-grained graphs, capturing the structural and semantic differences between code revisions. Then it leverages these representations to automatically generate human-readable commit messages. CCRRep compares itself to this model and was shown to achieve superior results.

Another significant contribution in this area is COMET, a method introduced by Mandli et al. [40], published in 2025 and therefore one of the most recent works included in the SR (which was concluded on January 31, 2025). COMET presents a novel CCR technique based on delta graphs, which explicitly encode differences between code versions, and integrates a quality assurance module with a transformer-based generator. According to the authors, COMET achieves state-of-the-art results in CMG. However, it is important to note that the authors did not compare their method to FIRA, citing difficulties in reproducing its implementation, nor did they compare it to CCRRep. This omission introduces uncertainty regarding the validity of their performance claims.

Another prominent downstream task investigated in the context of CCR is vulnerability detection. In this domain, the CodeJIT model proposed by Nguyen et al. [41] currently represents the most advanced approach, to the best of the author’s knowledge. CodeJIT introduces a graph-based multiview representation that captures both the modified and unmodified portions of the code in a commit. In doing so, it provides a more comprehensive context for detecting security vulnerabilities introduced by recent changes. The model outperforms earlier approaches such as LaPredict [42] and DeepJIT [6], both of which were also used as baselines in the evaluation of CCRRep. However, it is important to emphasize that CodeJIT addresses the task of vulnerability detection, while CCRRep is primarily evaluated for defect prediction. Although these two tasks share similarities, they are inherently different and are evaluated on different benchmark datasets, which limits the direct comparability of their results.

Other tasks, though explored less frequently than CMG and vulnerability detection, include JIT-DP and APCA. These tasks remain important, particularly for the evaluation of general-purpose CCR methods. Several models, including CCRRep, are evaluated in the context of JIT-DP to demonstrate the effectiveness of their learned representations in practical software engineering scenarios. Similarly, APCA has emerged as a useful task for assessing how well a model understands the semantics of code changes and their implications.

SRRQ3: *What data sets are employed in the learning of representations for code changes?*

During the SR, particular attention was paid to the datasets used in the development and evaluation of the CCR methods. Although a wide variety of datasets were identified in the reviewed literature, a number of them appeared consistently, being reused in multiple

studies. This recurrence indicates their perceived reliability and usefulness in benchmarking state-of-the-art solutions.

Of particular interest for this thesis is the collection of datasets utilized by the state-of-the-art CCR model [3], which also overlaps in part with the datasets used by CCAF [33]. Rather than relying on a single dataset, CCR aggregates several distinct datasets corresponding to different downstream tasks, including CMG, APCA, and JIT-DP. This multidataset approach enables comprehensive evaluation in a variety of use cases, increasing the generalizability of the results.

For the CMG task, two datasets are commonly used: one from the CoReC paper by Wang et al. [43], and the other from FIRA by Dong et al. [27]. For APCA, Lin et al. [19] introduced two data sets: CACHE-Small and CACHE-Large. The JIT-DP task is supported by a wider collection of datasets gathered by Zeng et al. [42], including project-specific data from Gerrit, Go, JDT, OpenStack, Platform, and Qt.

A brief summary of the datasets, including their size, programming language, input/output types, and associated task, is presented in Table 1.3.

Table 1.3. A brief summary of datasets.
Input/output specifies the type of input and output for the downstream task.

Task	Dataset	Language	Size	Input	Output
CMG	CoReC	Java	27,144	code	generation
	FIRA	Java	82,661		
APCA	CACHE-Small	Java	1,183	code, text	classification
	CACHE-Large	Java	49,694		
JIT-DP	Gerrit	Java	14,927	code	classification
	Go	Golang	19,009		
	JDT	Java	3,279		
	OpenStack	C++	22,757		
	Platform	Java	11,034		
	Qt	C++	23,912		

In summary, there are currently only a few general-purpose CCR methods capable of producing high-quality representations applicable across a wide range of downstream tasks. However, there is a noticeable trend toward the development of such versatile approaches. Among them, CCR [3] currently represents the state of the art. Due to its general applicability and strong performance in multiple evaluation settings, CCR will serve as the **baseline** for the method proposed in this thesis. This comparison will be made possible by using an overlapping set of benchmark datasets, ensuring a fair and consistent evaluation of performance.

1.2. Taxonomy of code representation techniques

In traditional ML approaches to the representation of source code, the focus was primarily on manually designing feature vectors extracted from the code. This process was laborious and required significant domain expertise to identify relevant syntactic and semantic features. Furthermore, such hand-made representations often suffered from poor generalizability. They were typically tailored to specific programming languages or problem domains, making it difficult to reuse them in different technological contexts or downstream tasks [12].

The emergence of DL, particularly the advent of transformer-based architectures, has significantly shifted the paradigm from manual feature engineering to automated representation learning. This shift led to the development of new approaches for encoding source code into input formats suitable for DL models. These learned representations tend to be more flexible, generalizable, and capable of capturing deeper semantic relationships within the code.

In the context of modern representation learning for code, we can identify three primary approaches to transforming code into machine-processable formats: lexical, tree-based, and graph-based representations. This classification has also been noted in many previous works [3, 12, 25, 27, 28, 32, 44]. Each of these categories emphasizes different structural or semantic aspects of code and is often selected based on the specific requirements of the downstream task or the nature of the information to be captured.

Across all of these representation types, there has been a growing interest in leveraging PTMs, which are commonly built using encoder-only transformer architectures. Decoder-only or encoder-decoder models are less frequently employed but can be useful in tasks such as code generation or translation. The use of PTMs allows for transfer learning and improves performance in a wide range of software engineering tasks, contributing to the broader applicability and robustness of AI models in source code analysis [5, 7].

The following subsections provide a detailed overview of each representation approach.

1.2.1. Lexical code representation

Lexical code representation focuses on the textual surface of the source code, specifically its words and symbols. During the SR, tokenization appeared to be the only lexical representation method identified.

A tokenizer processes the raw source code and splits it into a sequence of discrete units known as tokens. These tokens typically include keywords, identifiers, operators, punctuation symbols, and other syntactic elements. This approach is heavily influenced by natural language processing techniques, where similar strategies are used to process and model human language. Although there are several well-known tokenization algorithms,

such as SentencePiece [45], many research papers in the field do not explicitly state which tokenizer or tokenization strategy was used.

The tokenization process converts the source code from its textual form into a list of tokens. In standard tokenization schemes, each word and special character is treated as a separate token. This is illustrated in Listing 1.3, which shows the tokenized form of a sample Python function.

```
def func(x, y):
    i = x + y
    j = x - y
    if i == j:
        return i
    return 2 * i + j
```

Listing 1.2: A sample Python code snippet.

```
['def', 'func', '(', 'x', 'y', ')',
 ↪ ':', 'i', '=', 'x', '+', 'y',
 ↪ 'j', '=', 'x', '-', 'y', 'if',
 ↪ 'i', '=', '=', 'j', 'return',
 ↪ 'i', 'return', '2', '*', 'i',
 ↪ '+', 'j']
```

Listing 1.3: A tokenized code snippet from Listing 1.2.

The next step in tokenization is to transform the token sequence into a numerical format that can be used by machine learning models. This is typically achieved through a process known as embedding. Word embeddings are learned vector representations of tokens, where semantically similar tokens are mapped to vectors that are close in the embedding space. Popular embedding techniques, such as word2vec [46] and doc2vec [47], have been widely adopted and adapted for source code tasks. These methods help capture the semantic meaning of individual tokens and their contextual relationships.

Despite their effectiveness in modeling token-level semantics, lexical representations have a significant limitation: they fail to capture the structural properties inherent to source code, such as hierarchical syntax or data/control flow dependencies. Nevertheless, lexical representations remain attractive due to their simplicity and efficiency. Tokenization can typically be performed in linear time complexity $O(n)$, which may also be one reason why this mode of representation is so popular in the papers reviewed. [12, 44].

Moreover, tokenization is particularly well-suited for CCR, as it operates effectively even when applied to partial code fragments. This characteristic is especially important in scenarios where complete syntactic structures may not always be available. By focusing solely on local token sequences, lexical models can still extract meaningful patterns from code modifications without requiring a full understanding of the surrounding context.

1.2.2. Tree-based code representation

Tree-based code representation is a type of structural representation that emphasizes the hierarchical nature of source code. Among the approaches identified during the SR, the

AST was the only tree-based representation adopted. Its popularity is comparable to that of lexical tokenization, especially in tasks requiring structural code analysis [12].

An AST is a tree-like representation of the syntactic structure of the source code, where each node corresponds to a construct occurring in the code, such as variables, expressions, or control structures [48]. This representation captures the grammatical rules of the programming language and explicitly encodes the relationships between different code elements. By abstracting away from surface-level syntax and focusing on structural semantics, ASTs enable machine learning models to identify patterns that reflect the logical organization of code. This abstraction often reduces learning complexity and improves generalizability between languages and tasks [26, 44].

The AST corresponding to the code snippet shown in Listing 1.2 is illustrated in Figure 1.4. This visualization was made using the VisAST¹ Python package, which parses the source code and produces its tree representation in a structured format.

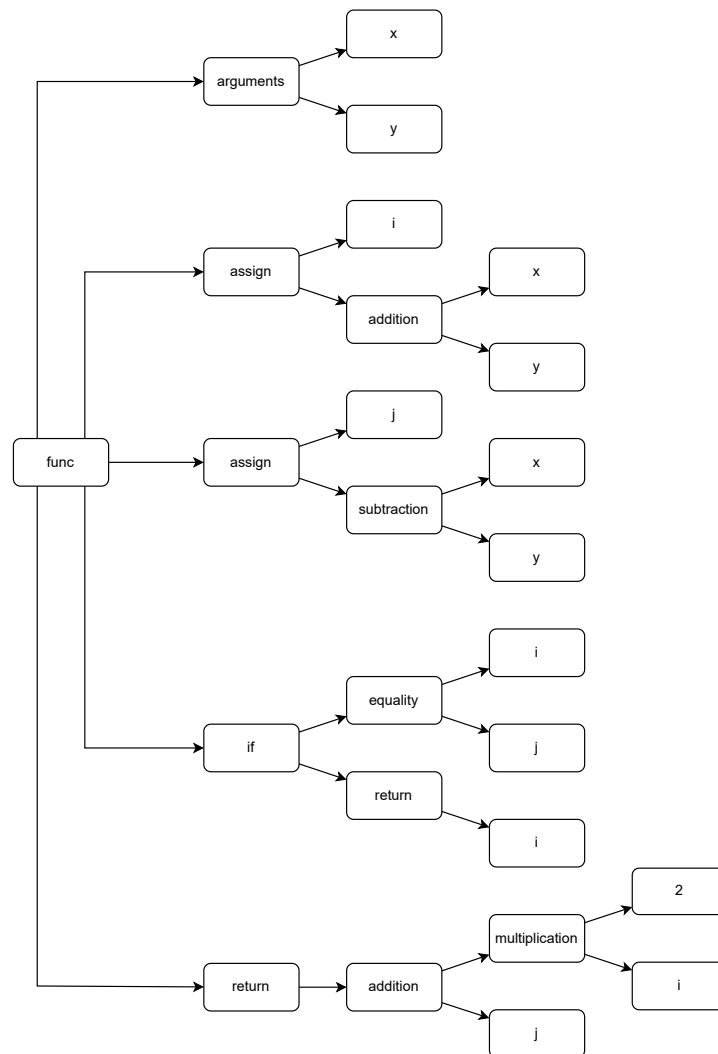


Fig. 1.4. Representation of the code snippet from Listing 1.2 using abstract syntax tree.
Own work.

¹ <https://pypi.org/project/VisAST/>

Despite their advantages, AST-based representations have some limitations when applied to CCR tasks. One of the key drawbacks is the need for access to a larger context of the source code. In many CCR scenarios, such as when analyzing commits or changes from version control systems, only partial information about code modifications is available. AST construction typically requires syntactically complete code, which makes it difficult or even impossible to generate meaningful trees from isolated diffs. Consequently, using ASTs for fine-grained change representation often involves non-trivial preprocessing steps, such as extracting and reconstructing larger code fragments surrounding the changes.

1.2.3. Graph-based code representation

Graph-based code representation refers to the transformation of source code from a textual form into a graph structure, where nodes and edges represent specific elements and relationships within the code. Although this approach is considerably less common than lexical (token-based) or tree-based representations such as AST, it is also the most diverse. A wide variety of graph types have been proposed and applied in the context of code analysis, making this category particularly rich in expressive power.

Among the graph representations most commonly used are the control flow graph [49] and the program dependency graph [50], both of which capture different aspects of the program semantics.

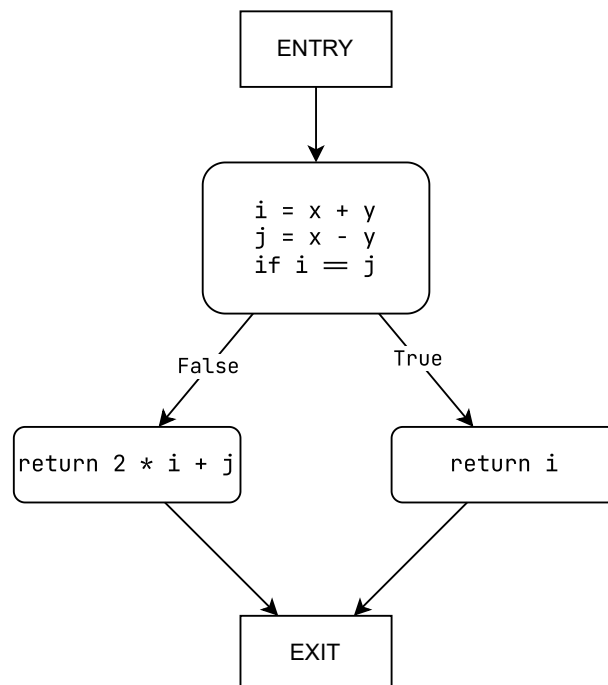


Fig. 1.5. Representation of the code snippet from Listing 1.2 using control flow graph.
Own work.

A control flow graph (CFG) models the order in which individual statements or code blocks are executed. In a CFG, the nodes represent instructions or control statements,

and the edges denote the possible paths that the execution of the program might follow [12]. Formally, a CFG is defined as a directed graph $g = (V, E)$, where V is the set of nodes representing basic blocks (e.g. functions, statements) and $E \subseteq V \times V$ is the set of directed edges representing the control flow between them. A basic block is a sequence of statements that is executed sequentially without interruption. An edge $e = v_{src} \rightarrow v_{dst}$ indicates that control can be transferred from block v_{src} to block v_{dst} [44].

An example CFG constructed from the Python code snippet shown in Listing 1.2 is presented in Figure 1.5.

A program dependence graph (PDG) represents both data and control dependencies between statements in a program [12]. Formally, it is a directed graph $g = (V, E)$ that captures which instructions depend on one another either due to shared data or due to control logic. The set of nodes V is divided into two categories: statement nodes V_{stmt} , which represent operations or instructions, and predicate nodes V_{pred} , which represent conditional expressions evaluating true or false. The edges E are also divided into two types: control dependency edges E_c and data dependency edges E_d .

A control dependency edge $e_c = v_{src} \rightarrow v_{dst}$ indicates that the execution of node v_{dst} depends on the evaluation of the trueness of a predicate v_{src} . A data dependency edge $e_d = v_{src} \rightarrow v_{dst}$ denotes that node v_{dst} uses data defined or modified by node v_{src} [44].

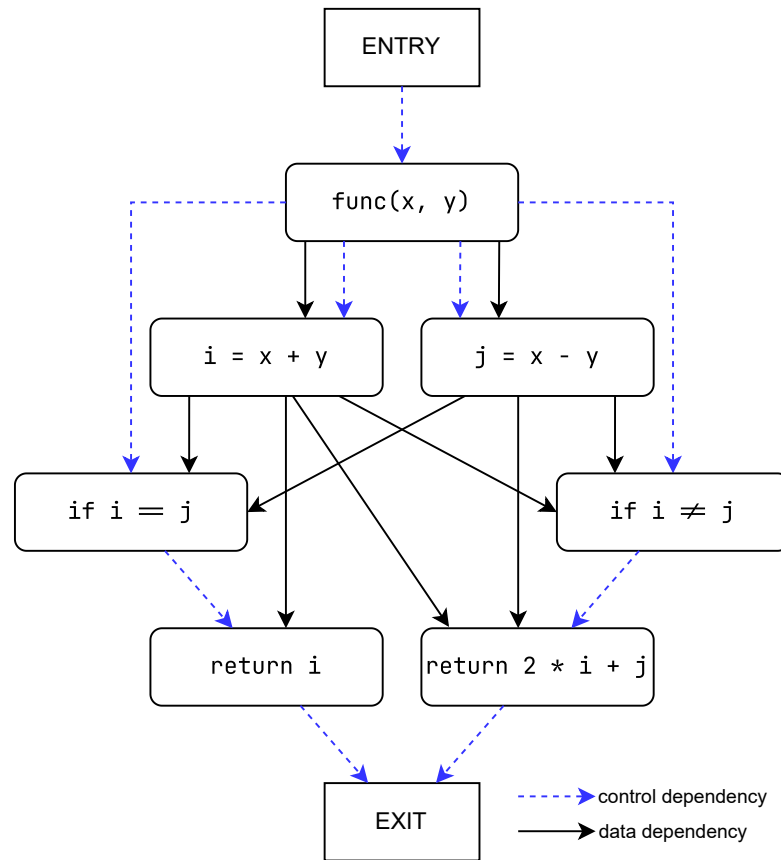


Fig. 1.6. Representation of the code snippet from Listing 1.2 using program dependence graph. Own work.

Figure 1.6 illustrates the PDG for the code snippet from Listing 1.2, showing both types of dependencies as part of the graph structure.

Despite their expressiveness, graph-based representations face significant challenges when applied to CCR tasks. Like ASTs, graph-based models typically require access to larger and syntactically complete code contexts. However, many CCR applications – especially those that analyze diffs or commit-level changes – operate on incomplete or isolated code fragments. As a result, generating valid and meaningful graphs from such partial inputs can be highly non-trivial and may require extensive preprocessing, such as extracting and reconstructing larger code fragments surrounding the changes.

Additionally, graph-based representations are often more complex to implement and interpret than lexical or tree-based approaches. They demand sophisticated parsing, graph building, and transformation tools, which can increase computational cost and development effort. However, this complexity also brings flexibility and power, as the vast number of possible graph structures allows for fine-grained modeling of various aspects of code behavior and semantics.

2. Proposed solution and its implementation

This chapter presents the implementation of the proposed solution, which consists of two main modules. It describes the developed components and explains the approaches applied to achieve the thesis objective – learning representations of code changes using an artificial intelligence model.

2.1. System infrastructure

The research infrastructure developed for this thesis consists of two main modules. The first module is dedicated to experiments involving the query back mechanism, originally proposed by Liu et al. [3] in their work on CCRep. It is based on and extends the official reproduction package provided by the authors. The second module is an entirely original, from scratch implementation of a framework designed to explore a retrieval augmentation (RA) based approach. In accordance with the recommendations of Madeyski et al. [51] on reproducible research, both modules are made available as open source at <https://github.com/jakubzehner/MasterThesis>. This decision addresses multiple threats to the validity of the research by enabling full transparency and reusability of the experiments carried out.

2.1.1. Query back experimentation module

This module is built upon the original implementation of CCRep [3], written in Python, and relies on the AllenNLP [52] library, which was already unsupported at the time of CCRep’s publication. Additionally, the reproduction package provided by the original authors included an incomplete and outdated list of library dependencies. As a result, the reproduction process required significant manual effort, including resolving version conflicts and compatibility issues.

To mitigate such reproducibility challenges in the future and facilitate consistent execution in different environments, an Apptainer [53] container image was created. The apptainer is a containerization technology specifically recommended for high performance computing (HPC) environments and ensures the portable and isolated execution of scientific workflows.

Modifications made to this module focused on two main areas. First, the set of evaluation metrics was significantly extended. The original implementation included only

a minimal set of metrics, which made it difficult to perform a meaningful comparison between approaches. Given that CCRRep represents a state-of-the-art solution in the field of CCR, the ability to perform a comprehensive comparison was considered crucial.

Second, the module was adapted to support a wider range of pre-trained encoders, including those based on encoder-decoder architectures used in modern Large Language Models (LLMs). This enhancement was essential for conducting broader and more flexible experiments with various foundation models, which play a central role in current software engineering research.

2.1.2. Retrieval augmentation experimentation framework

The second module, referred to as RAFramework, was implemented entirely from scratch to support experimentation with the retrieval augmentation (RA) approach in the context of code change representation. Python was chosen as the primary language due to its widespread adoption in modern AI research and development, as well as the availability of state-of-the-art libraries such as PyTorch [54] and Hugging Face Transformers [55]. Python also allows researchers to focus on experimentation rather than low-level implementation details.

RAFramework was designed to address reproducibility shortcomings observed in previous work [3, 33]. To that end, the framework employs a modern and deterministic package management system called *uv*¹, which ensures reproducible environments and streamlined dependency resolution. The quality and consistency of the code are maintained through the use of *ruff*², a tool that combines a formatter and a linter.

The framework makes use of robust and well-established libraries such as NumPy [56], scikit-learn [57], and Matplotlib [58] to facilitate numerical computation, model evaluation, and result visualization.

RAFramework is designed with modularity in mind and introduces a layer of abstraction to simplify extensibility. This design is illustrated in the class diagram presented in Figure 2.2. For clarity, private fields and methods have been omitted. The framework operates in two main stages. The first stage, triggered by the `EncodeDatasets` module, is responsible for generating code embeddings using any selected encoder. These embeddings are then cached to disk. This design choice significantly speeds up subsequent experiments, as the solution assumes no fine-tuning of the selected encoder models. Consequently, the embeddings remain constant across runs and can be reused safely.

The encoding process relies on an abstract class called `GenericReader`, which provides a unified application programming interface (API) for the access of the data set. Specific implementations of this reader handle preprocessing based on the characteristics of the selected dataset, ensuring consistent input for the embedding stage.

¹ <https://github.com/astral-sh/uv>

² <https://github.com/astral-sh/ruff>

The second stage involves training and evaluation of models and is also based on abstract base classes that expose a consistent API. These base classes can be extended to implement a wide range of RA-based approaches. Key components include `BinaryClassifierBase` and `RetrieverBase`, the latter working in conjunction with the `RetrievalAugmentationDB`. These components are extended by different variants of the RA method, described in greater detail in Section 2.4.

To ensure the reliability of the evaluation, the framework includes cross-validation mechanisms, which help produce statistically significant results [59]. In addition, an early stop mechanism is implemented to prevent overfitting, which is particularly important when working with limited data sets or complex models [60, 61]. The evaluation of trained models is performed through a dedicated module that wraps implementations of scikit-learn metrics [57] as extensions of the `MetricBase` class. This design enables task-independent evaluation by allowing reuse of the same evaluation module while simply swapping in task-specific metric modules.

An overview of the structure of the `RAFramework` package is presented in Figure 2.1, which illustrates the organization of the components of the framework at the directory level.

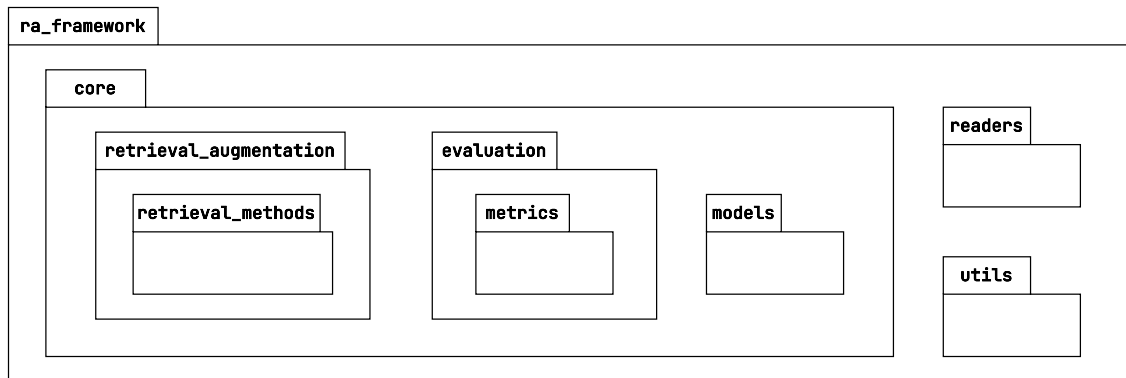


Fig. 2.1. Package diagram of `RAFramework`.
Own work.

2.2. Use of pre-trained models

A crucial component of both experimental modules presented in this thesis is the utilization of LLMs, specifically their pre-trained encoder components. These pre-trained models constitute the core of state-of-the-art solutions to generate high-quality CCR [3, 33].

The first experimental module builds on the approach proposed in `CCRep` [3], where a PTM is fine-tuned and combined with a query back mechanism to improve the quality of code change representations. Although the `CCRep` framework is theoretically encoder-agnostic, its implementation is tailored exclusively for CodeBERT [14]—a widely adopted foundational model in software engineering research. CodeBERT consists of 125

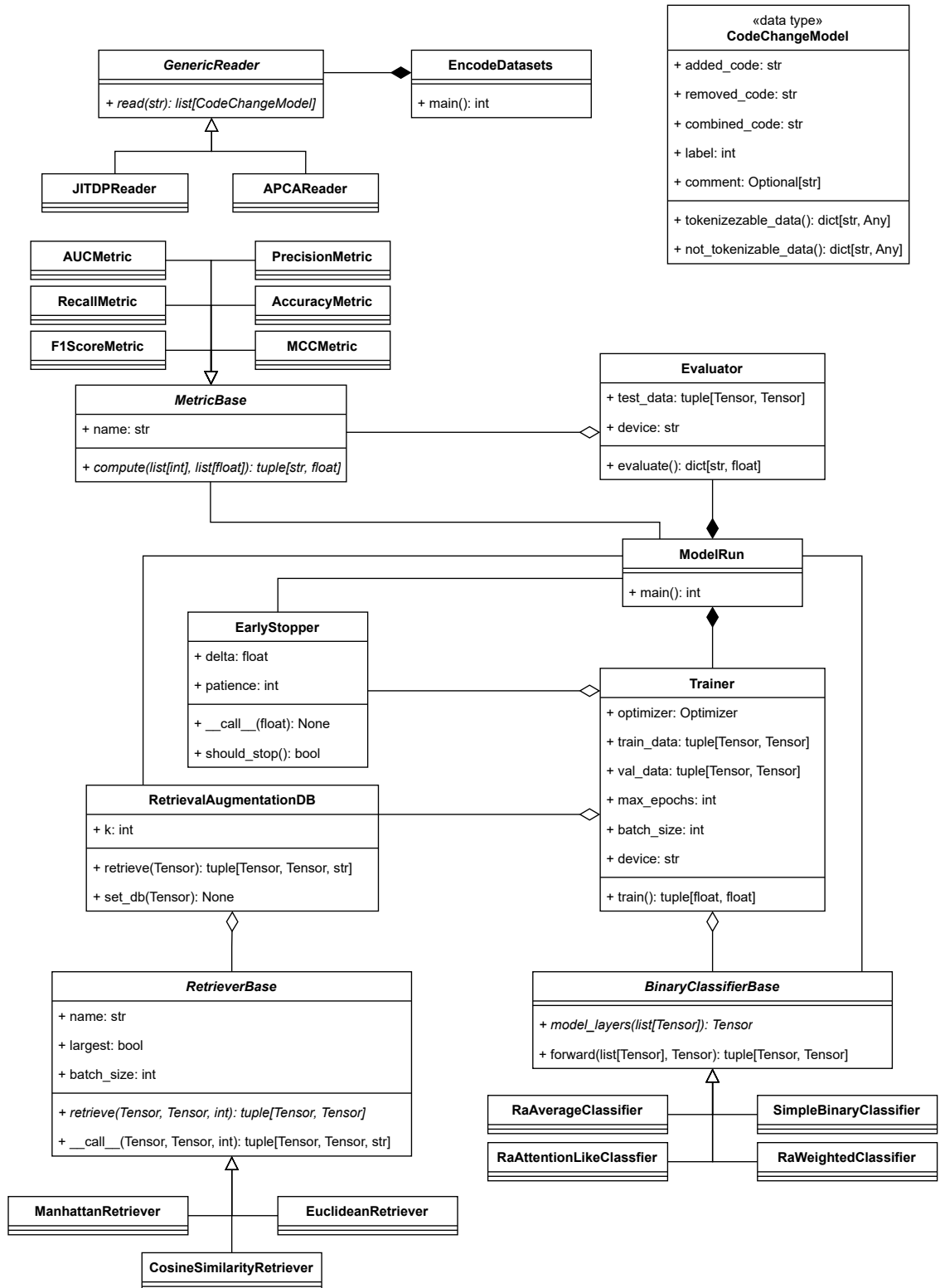


Fig. 2.2. Class diagram of RAFramework.
Own work.

million parameters and is pre-trained on a mix of source code and natural language data, making it a suitable starting point for fine-tuning tasks related to source code understanding.

In this thesis, the original CCRep implementation was extended to support additional encoder architectures beyond CodeBERT. These include:

- GraphCodeBERT [15], which integrates data flow graph information [62] during pre-training to enhance the model’s structural understanding of code, and also consists of 125 million parameters.
- UniXcoder [39], a cross-modal model capable of processing both code and natural language modalities, with 125 million parameters as well.

Furthermore, the CCRep-based module was adapted to support encoder-decoder architectures. In such models, the encoder and decoder components are typically used together in tasks such as translation or summarization. However, for the purpose of extracting code embeddings, only the encoder portion is required. To utilize these models, the encoder and decoder had to be explicitly separated. For generating code embeddings, the average of the encoder’s output vectors across all input tokens was used. This technique is supported by the findings of Ni et al. [63], who evaluated several strategies for sentence-level embedding generation in transformer-based models such as Sentence-T5 [64]. Their results demonstrated that averaging all token-level encoder outputs provides superior performance compared to other methods, such as using the first token output or the decoder first-layer output.

Following this approach, encoder-decoder models from the CodeT5 family were incorporated:

- CodeT5 [16], with 220 million parameters in total.
- CodeT5+ [65], an enhanced version of CodeT5, considered one of the current state-of-the-art encoder-decoder models for code-related tasks. Two versions of this model were used: the 220M and 770M parameter variants.

Importantly, in these configurations, encoder components account for approximately half of the total parameter count, 110M and 335M, respectively.

It is worth noting that CodeT5+ is also available in significantly larger versions (2B, 6B, 16B parameters). However, these larger variants adopt a different architecture, characterized by a "shallow encoder and deep decoder" design [66]. Such an architecture makes them unsuitable for embedding-focused tasks, as the encoder is comparatively underpowered. Consequently, only the smaller and more balanced versions of CodeT5+ were used in this study.

Despite its conceptual elegance and performance, the CCRep approach comes with a significant computational cost. Fine-tuning even a midsized model, such as CodeBERT, is a resource-intensive process that can take several hours on top-tier GPUs optimized

for machine learning workloads. This limitation poses a barrier to the scalability and repeatability of the experiments.

To address this challenge, an alternative solution based on RA was explored. This approach, implemented in the RAFramework introduced earlier, leverages pre-trained encoders without performing any fine-tuning. As a result, computational requirements are significantly reduced. For RA-based experiments, the encoder component of the CodeT5+ model parameter 770M was selected, corresponding to approximately 335 million parameters. The same embedding strategy was applied here, averaging the encoder output across all input tokens, in line with the best practices discussed earlier.

2.3. Approach based on the query-back mechanism

The approach developed in the first module of this thesis is based on the method proposed in CCRep [3]. The overall architecture of the solution is illustrated in Figure 2.3 and consists of several key stages.

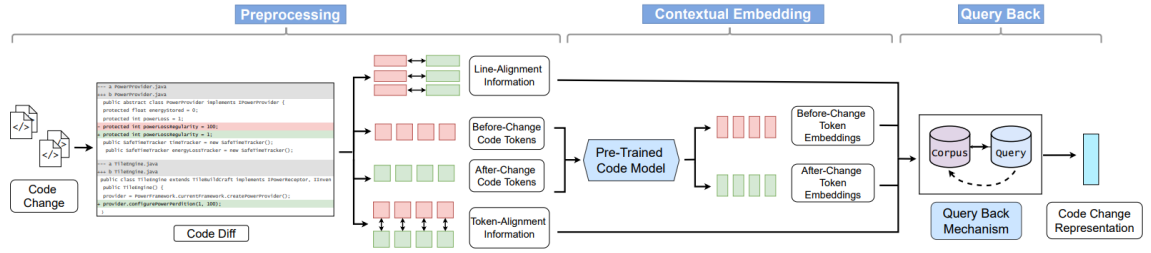


Fig. 2.3. The overall framework of CCRep.
Source: CCRep, Liu et al. [3]

The first stage is pre-processing, where the changes in the raw code are transformed using a specialized tool. This tool decomposes each code change into structured fragments that consist of added lines, removed lines, and surrounding unchanged context lines (i.e., lines appearing before and after the change). In addition to line-level decomposition, the tool identifies and aligns modified tokens and modified lines across versions of the code. This preparation step ensures that subsequent stages can work with a structured representation of the code modifications.

The second stage is contextual embedding, in which the previously extracted code fragments – both additions and deletions – are passed through a pre-trained encoder to generate meaningful vector representations (embeddings). These embeddings capture the syntactic and semantic characteristics of the code snippets. In this work, various encoder architectures are used for this purpose, as detailed in Section 2.2.

Following the embedding step, the query back mechanism is applied. This mechanism, introduced by the original authors of CCRep, plays a central role in enhancing the final CCR by selectively retrieving and aggregating the most relevant contextual information.

The key idea behind this mechanism is to treat the modified code fragments as queries that are used to retrieve informative fragments from both the pre-change and post-change versions of the code. These retrieved fragments are then integrated to build the final representation of the change.

More specifically, the mechanism takes as input both the embeddings and structured information about token-level and line-level changes. It generates a query vector from the modified fragment, which is then used to attend to the surrounding context to extract the information that is most relevant to understanding the nature of the change. The authors proposed two versions of this mechanism: one that operates at the token level and another that operates at the line level. In addition, a hybrid version was introduced that combines both granularities to capture complementary information from each perspective.

The output of this stage is a comprehensive, information-rich representation of the code change, designed to facilitate downstream tasks.

2.4. Approach based on retrieval augmentation

The approach proposed in the second module of this thesis is inspired by the Retrieval-Augmented Generation (RAG) method introduced by Lewis et al. [67], which aims to improve text generation by incorporating external knowledge into the generation process, thereby reducing hallucinations and producing more relevant and specific outputs. A brief review of the literature revealed that several studies have adapted this approach to classification tasks in both the textual and visual domains [68–70]. However, the application of RA in this context remains underexplored. To our knowledge, supported by a systematic review of the literature presented in Section 1.1, this is the **first work to investigate RA in the field of CCR**.

An overview of the proposed approach is illustrated in Figure 2.4, and consists of the following steps.

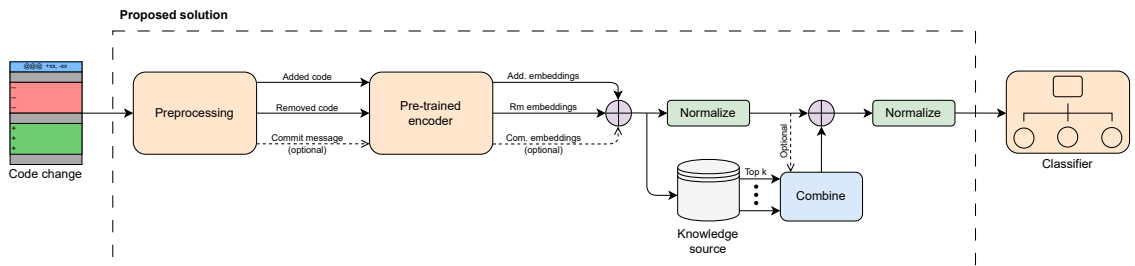


Fig. 2.4. Overview of the proposed RAframework.
Own work.

The first step in the proposed solution is preprocessing. At this stage, code changes are analyzed to extract added and removed lines of code. If available in the dataset, commit

messages are also extracted. These elements are treated as individual text strings and passed forward in the pipeline.

The next step involves encoding the extracted text using a pre-trained encoder. Specifically, we used the encoder from the CodeT5+ model with 770 million parameters, of which the encoder itself contains approximately 335 million parameters. Input sequences are first tokenized using the CodeT5+ tokenizer, which supports input lengths of up to 512 tokens. Following prior work [3, 33], sequences longer than this limit are truncated, while shorter ones are padded with zeros to the required length. The tokenized sequences are then passed through the PTM, which outputs a sequence of embeddings. Subsequently, these are averaged to produce a fixed-size vector, as described in Section 2.2. The resulting embeddings are floating point vectors of size 1024, reflecting the dimensionality of the selected encoder.

The individual embeddings are then combined to form a unified representation:

$$E = E_a \oplus E_r \quad \text{or} \quad E = E_a \oplus E_r \oplus E_c$$

where E is the combined embedding, E_a represents the embedding of added lines of code, E_r the embedding of removed lines, and E_c the embedding of the commit message (if available).

This embedding E is used as a query to search a knowledge base, which consists of all embeddings from the training set of the given data set. During training, the embedding used as a query is excluded from the knowledge base to prevent data leakage and reduce overfitting. Multiple retrieval strategies for selecting the top- k most relevant embeddings are implemented, as detailed in Section 2.4.1. The retrieved embeddings serve as contextual information intended to augment the original input.

The selected embeddings are passed through a *combine* layer, which fuses the retrieved information using one of the three methods described in Section 2.4.2. The output of this layer is combined with the normalized version of the original embedding vector E , as follows:

$$E_f = \hat{E} \oplus E_{com}$$

where E_f is the final embedding, E_{com} is the result of the *combine* layer, and $\hat{E}$ is the normalized form of E . The normalization process is defined as follows. Let the embedding vector be:

$$E = [E_1, E_2, \dots, E_d] \in \mathbb{R}^d$$

Then the normalized vector $\hat{E}$ is given by:

$$\hat{E}_i = \frac{E_i - \mu}{\sqrt{\sigma}} \quad \text{for } i = 1, 2, \dots, d$$

where:

$$\mu = \frac{1}{d} \sum_{i=1}^d E_i, \quad \sigma = \frac{1}{d} \sum_{i=1}^d (E_i - \mu)^2$$

Finally, the vector E_f is also normalized, resulting in $\hat{E}_f$, which serves as the output representation of the CCR model. For downstream tasks, following prior work [3, 33], this embedding is passed to a task-specific layer, in this case it is a simple classifier composed of a single layer perceptron with sigmoid as activation function, responsible for the final prediction.

2.4.1. Methods of retrieval embeddings

The retrieval component of the proposed framework is responsible for selecting the top- k most relevant embeddings from the knowledge base, based on a given query embedding. This process is crucial for the quality of the final representation, as the relevance and informativeness of the retrieved vectors directly impact the downstream performance of the classifier.

To ensure generalizability and allow empirical evaluation, three distance- or similarity-based strategies were implemented: cosine similarity, Euclidean distance, and Manhattan distance. These approaches are well-established in the field of information retrieval and vector similarity search.

Cosine similarity.

Cosine similarity measures the cosine of the angle between two nonzero vectors, which represents their orientation in the vector space, regardless of their magnitude. It is defined as:

$$S_C(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

where $A \cdot B$ is the dot product of the vectors A and B , and $\|A\|$, $\|B\|$ are their respective Euclidean norms. Cosine similarity is particularly effective in sparse and high-dimensional vector spaces and has been widely used in natural language processing to compare text or sentence embeddings [71].

Euclidean distance.

The Euclidean distance between two vectors is the straight-line distance (L2 norm) between their coordinates in the embedding space:

$$dist_{euc}(A, B) = \|A - B\|_2$$

This metric takes into account both direction and magnitude and is sensitive to the scale of the data. In our setup, the top- k vectors with the smallest Euclidean distances to the

embedding of the query are selected.

Manhattan distance.

Also known as the L1 norm or taxi distance, the Manhattan distance measures the sum of the absolute differences between the corresponding elements of two vectors:

$$dist_{man}(A, B) = \|A - B\|_1$$

This distance metric is less sensitive to outliers than the Euclidean distance and can sometimes offer improved performance in high-dimensional spaces.

2.4.2. Methods of combining embeddings

Once the top- k most relevant embeddings have been retrieved, they must be integrated into a single vector that represents contextual knowledge that should be fused with the original input embedding. This process is carried out by the combine layer, for which three distinct strategies have been implemented: arithmetic average, weighted average, and an attention-inspired mechanism.

Arithmetic Average (RA-avg).

The simplest combination strategy computes, as illustrated in Figure 2.5, the mean of all retrieved embeddings. Let $R = \{R_1, R_2, \dots, R_k\}$ be the set of retrieved embeddings, each of dimension d . The combined vector is calculated as:

$$E_{com} = \frac{1}{k} \sum_{i=1}^k R_i$$

This approach assumes an equal contribution from all the vectors retrieved. Although computationally efficient, it does not take advantage of retrieval scores, which could encode valuable relevance information. However, this method is a strong baseline and serves as a useful point of comparison for more sophisticated approaches.

Weighted Average (RA-weight).

To incorporate the retrieval scores into the fusion process, the second strategy applies a weighted average, as shown in Figure 2.6. Let w_i denote the relevance score for the i -th retrieved vector. The final combination is given by the following.

$$E_{com} = \frac{\sum_{i=1}^k w_i \cdot R_i}{\sum_{i=1}^k w_i}$$

This method ensures that the more relevant vectors have a larger influence on the final

representation. When distance metrics are used, the scores are inverted and normalized to ensure proper weighting.

Attention-Inspired Mechanism (**RA-att**).

The last method shown in Figure 2.7 adapts the attention mechanism introduced by Vaswani et al. [13] to the retrieval setting. In traditional self-attention, the output is a weighted combination of value vectors, where the weights are determined by the similarity between queries and keys. In our adapted version, the original embedding E serves as the query Q , while the retrieved embeddings act as both keys K and values V . The attention computation is as follows:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d}})V$$

In our case, since Q is a single vector of shape $(d,)$ and K, V are matrices of shape (k, d) , this produces a single output vector of shape $(d,)$, representing the contextually weighted average of the retrieved vectors. This mechanism allows the model to learn how much attention to pay to each retrieved vector based on its compatibility with the input embedding.

To ensure compatibility (already taking into account batch size), Q is reshaped from $(batch_size, d)$ to $(batch_size, 1, d)$, while retrieved from knowledge base vectors form a tensor of shape $(batch_size, k, d)$. The final output has shape $(batch_size, d)$ and is passed forward in the pipeline.

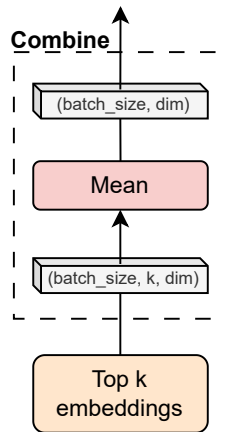


Fig. 2.5. Visual representation of RA-avg as a combine layer.
Own work.

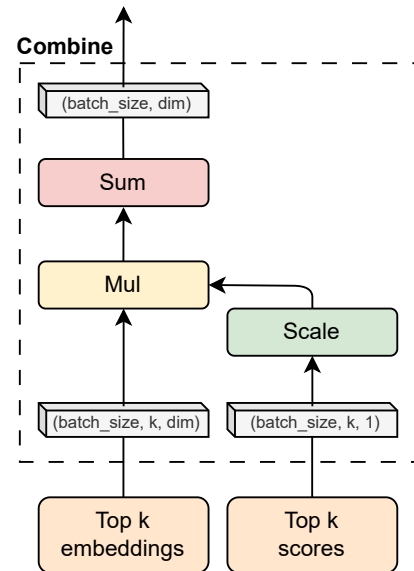


Fig. 2.6. Visual representation of RA-weight as a combine layer.
Own work.

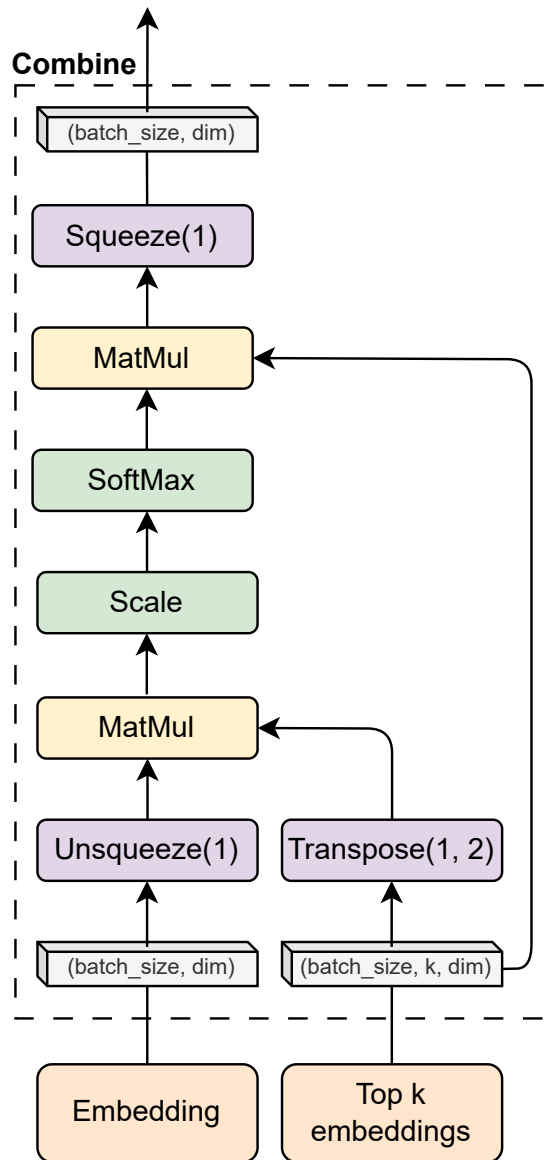


Fig. 2.7. Visual representation of RA-att as a combine layer.
Own work.

3. Empirical evaluation of the proposed solution

This chapter is dedicated to the empirical evaluation of the proposed solutions. It begins by describing the downstream tasks used to assess the quality of the learned CCR. These tasks serve as practical benchmarks to verify how well the representations capture meaningful aspects of code modifications. Next, the chapter outlines the selected datasets and evaluation metrics used to measure the performance of the models. Furthermore, it presents the research questions that guide the evaluation process and provides analysis of the obtained results.

3.1. Downstream tasks

Following prior work such as CCRRep [3] and CCAF [33], three downstream code-related tasks were used to evaluate the proposed solutions. For the first module, an extension of the CCRRep framework, three tasks were selected: commit message generation, automated patch correctness assessment, and just-in-time defect prediction. In contrast, the second module, called RAFramework, was evaluated using two tasks – automated patch correctness assessment and just-in-time defect prediction.

The task of commit message generation (CMG) [43] is highly relevant from a software engineering perspective, as it aims to automate the process of writing commit messages, an essential part of daily development workflows. High-quality commit messages facilitate a better understanding of code changes, improve collaboration, and support maintenance activities. However, developers often lack the time or motivation to write informative commit messages[20], which results in inconsistent or insufficient documentation. The CMG task involves automatically generating a concise and accurate summary of code changes, based solely on the code diff.

Automated patch correctness assessment (APCA) [19] is a task associated with the broader field of automated program repair (APR) [72]. Its objective is to determine whether a given code patch correctly fixes a bug. Many tools in this domain automatically generate candidate patches and validate them using test suites. However, a patch that passes all tests is not necessarily correct, which is commonly referred to as overfitting to the test suites [73]. To address this issue, researchers proposed APCA as a binary classification task that evaluates whether a patch is truly correct, beyond just validation of the test suite. This makes it an important step toward more reliable automated debugging tools.

The third task, just-in-time defect prediction (JIT-DP) [42], is one of the most extensively studied problems in software quality assurance. Since bugs are an inevitable part of software development and are often detected only in later stages, they tend to incur significant maintenance costs as the codebase grows [6]. JIT-DP aims to predict whether a given code change introduces a defect, enabling early detection before the code is merged or deployed. This can significantly reduce the effort required for code reviews and improve overall software reliability. The task is typically formulated as a binary classification problem, where each code change is labeled as either defect-inducing or clean.

Together, these downstream tasks provide a comprehensive evaluation framework for assessing the quality and practical utility of learned code change representations. Each task captures a different dimension of software development challenges, and success in these tasks reflects the model’s ability to understand and reason about code modifications.

3.2. Overview of selected datasets

To ensure a fair and meaningful comparison with current state-of-the-art methods, this study uses publicly available datasets that were originally collected — though not created — by the authors of CCRRep [3]. These datasets have been widely adopted in several influential studies and are considered of high quality. An important advantage of using these datasets is that, with the exception of the APCA dataset (due to the nature of the task), they are based on real-world data rather than synthetically generated samples. Specifically, they are derived from mature open source projects with large codebases and substantial contributor activity, which ensures the practical relevance and robustness of the experimental setup.

Table 3.1. Summary of datasets used for downstream tasks.
Input/output specifies the type of input and output for the downstream task.

Task	Dataset	Language	Size	Input	Output
CMG	FIRA	Java	82,661	code	generation
APCA	CACHE-Small	Java	1,183	code, text	classification
JIT-DP	Gerrit	Java	14,927		
	Go	Golang	19,009	code	classification
	JDT	Java	3,279		

As presented in Table 3.1, for the CMG task, the FIRA dataset [27] was used. It contains more than 82,000 data points, each representing a code change and its associated commit message. According to the authors of the dataset, FIRA is constructed from commits taken from the top 1,000 most popular Java repositories on GitHub, excluding rollback commits, merge commits, and duplicate entries. The large size and diversity of

this dataset make it suitable for training and evaluating models aimed at summarizing code changes.

For the APCA task, the CACHE-Small dataset [19] was used. This dataset is significantly smaller, containing 1,183 labeled instances. It is made up of patches derived from the Defects4J benchmark [74], a widely recognized dataset for evaluating automated program repair (APR) tools. All the code in CACHE-Small is written in Java, reflecting the fact that most APR systems have been evaluated on Java-based projects. The dataset is balanced, containing 535 correct patches (45%) and 648 incorrect ones (55%), allowing a reliable binary classification task setup.

For the JIT-DP task, three datasets of varying sizes were selected: Gerrit, Go, and JDT. All of them originate from open source projects in the real world. The Gerrit and JDT datasets are based on Java projects, while the Go dataset is derived from a project written in Golang. These datasets were initially collected by Zeng et al. [42] to evaluate deep learning-based defect prediction models. The Gerrit dataset is highly imbalanced, consisting of 14,927 code changes, of which only 1,781 (12%) are defect-inducing. In contrast, the Go and JDT datasets are more balanced: Go contains 19,009 changes with 8,145 (43%) labeled buggy, while JDT includes 3,279 changes with 1,602 (49%) introducing defects.

These datasets were chosen to reflect a variety of task types and codebases, providing a robust foundation for evaluating the performance and generalizability of the proposed code-change representation models.

3.3. Evaluation metrics

To comprehensively assess model performance across various downstream tasks, a set of evaluation metrics was used that was specifically tailored to the nature of each task type: binary classification and text generation. The selection of metrics was intentionally extended beyond those used in the state-of-the-art baseline solution, CCRep [3], which, in the most extreme case (JIT-DP task), reports only a single metric, AUC. Although useful, the AUC does not take into account precision or negative predictive value. As highlighted in Chicco and Jurman [75] work, this can lead to overly optimistic performance estimates, potentially distorting the actual effectiveness of the model.

3.3.1. Evaluation metrics for binary classification tasks

For binary classification tasks, namely APCA and JIT-DP, a total of six metrics were used to provide a more holistic evaluation: the commonly used accuracy, precision, recall and F1 score, as well as two additional and more robust metrics. Matthews correlation coefficient (MCC) [76] and area under the receiver operating characteristic curve (AUC) [77].

This selection was motivated by the need to fairly evaluate models even under conditions of class imbalance.

The following metrics were calculated based on the standard classification outcomes (in addition to the AUC, which uses the probability that it will be a given class):

- TP : true positives,
- FP : false positives,
- TN : true negatives,
- FN : false negatives.

Accuracy.

Accuracy measures the proportion of all correct predictions (both positive and negative) out of the total number of predictions made. Although widely used, accuracy can be misleading in unbalanced datasets, where it may favor the majority class.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN}$$

Precision.

The precision indicates how many of the instances predicted as positive are actually positive. It is particularly important in applications where false positives are costly, as it reflects the model's ability to avoid incorrectly labeling negative examples as positive.

$$Precision = \frac{TP}{TP + FP}$$

Recall.

Recall, also known as sensitivity or true positive rate, measures how many of the actual positive instances the model correctly identified. High recall means that the model captures most positive cases, but may come at the cost of more false positives.

$$Recall = \frac{TP}{TP + FN}$$

F1 score.

The F1 score is the harmonic mean of precision and recall. It provides a balanced measure that takes into account both false positives and false negatives, making it particularly useful when dealing with uneven class distributions.

$$F_1 = \frac{2TP}{2TP + FN + FP}$$

Matthews Correlation Coefficient (MCC).

MCC is a more informative and reliable metric for binary classification, especially in the presence of a class imbalance. Returns a value between -1 and +1, where +1 indicates

perfect prediction, 0 indicates random prediction, and -1 indicates total disagreement between prediction and reality.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Area Under the ROC Curve (AUC).

AUC evaluates the model’s ability to discriminate between positive and negative classes across various threshold values. A higher AUC value indicates better performance. However, the AUC should be interpreted cautiously, as it may still be high even if the model performs poorly in the minority class.

3.3.2. Evaluation metrics for text generation task

For the CMG task, which involves generating natural language text based on code changes, five well-established metrics were used to assess the quality of generated text: B-Norm BLEU [78], METEOR [79], ROUGE-1, ROUGE-2, and ROUGE-L [80]. These metrics are commonly used in natural language generation and machine translation tasks to measure various aspects of similarity between the generated text and reference (ground truth) output.

B-Norm BLEU.

BLEU (Bilingual Evaluation Understudy) is a precision-based metric that measures how many n-grams (contiguous sequences of words) in the generated text match those in the reference text. The B-Norm BLEU variant applies normalization to improve comparability across different datasets. Although BLEU emphasizes exact word matches, it does not account for synonymy or semantic similarity.

METEOR.

Metric for evaluation of translation with explicit ordering is a recall-oriented metric that also considers stemming and synonym matches, providing a more semantically informed measure of similarity between texts. It often correlates better with human judgment than BLEU, especially in tasks involving natural language generation.

ROUGE-1

This metric measures the overlap of unigrams (individual words) between the generated text and the reference. Captures basic word-level similarity and is particularly useful for assessing content adequacy in short summaries.

ROUGE-2

It is an expansion of ROUGE-1 considering the overlap of the bi-gram (two-word sequence). This metric evaluates the fluency and local coherence of the generated text, as matching word pairs are indicative of correct phrasing.

ROUGE-L

This metric computes the longest common subsequence between the generated and reference texts. It is sensitive to sentence-level structure and assesses how well the generated text captures the overall sequence of concepts from the reference.

3.4. Research questions

This thesis aims to address two main research questions (RQs), each further divided into several sub-questions. These questions are closely tied to the implementation of two distinct modules developed in this work. The first research question refers to the modification and evaluation of the current state-of-the-art solution, CCRRep [3]. The second question concerns the newly proposed RAFramework module, which introduces a novel approach to the problem of CCR based on RA.

The first research question is formulated as follows.

RQ1: *How does the use of various pre-trained encoders, in combination with a query back mechanism, affect the quality of code change representations?* This question is further divided into two sub-questions:

RQ1.1: *How does the use of different pre-trained encoders affect the quality of code change representations in binary classification tasks?* This sub-question investigates whether any of the selected pre-trained encoders generate significantly better embeddings when used together with the query back mechanism in binary classification tasks such as APCA and JIT-DP. The encoders evaluated in this context include CodeBERT, Graph-CodeBERT, UniXcoder, CodeT5, and CodeT5+, all of which are described in detail in Section 2.2.

RQ1.2: *How does the use of different pre-trained encoders affect the quality of code change representations in text generation tasks?* This sub-question is conceptually similar to RQ1.1 but focuses on a different type of task, code change message generation (CMG). The pre-trained encoders tested for this task are UniXcoder and CodeT5+.

The second research question explores a novel approach introduced in this work:

RQ2: *How does applying a novel retrieval augmentation technique, instead of fine-tuning pre-trained encoders, impact the quality of code change representations?* This question is also divided into several sub-questions to allow a more thorough analysis of the proposed approach.

RQ2.1: *How does the number of feature vectors retrieved affect the quality of code change representations?* This sub-question focuses on evaluating the impact of the number

of knowledge vectors retrieved from the external memory, as described in Section 2.4. The experiments are carried out on the APCA task using four different top- k values for the number of retrieved vectors: 3, 5, 7, and 10. Each top- k configuration is evaluated across all combinations of the retrieval methods and vector combination strategies implemented, resulting in 9 configurations per top- k value.

RQ2.2: *How does the choice of retrieval method affect the quality of code change representations?* This sub-question aims to determine which of the three implemented retrieval techniques, cosine similarity, Euclidean distance, or Manhattan distance (described in Section 2.4.1) provides the most relevant feature vectors. The evaluation is performed on the APCA task across all combinations of top- k values specified in RQ2.1 and vector combination methods, resulting in 12 configurations per retrieval method.

RQ2.3: *How does the method of combining retrieved feature vectors affect the quality of code change representations?* The goal of this sub-question is to identify the most effective approach for aggregating the retrieved knowledge vectors into a final representation of the code change. The evaluated methods are RA-avg, RA-weight, and RA-att, each described in Section 2.4.2. These models are tested in both the APCA and JIT-DP tasks using cosine similarity-based retrieval and a fixed top- $k = 5$.

All stated RQs are summarized along with the downstream tasks used in Table 3.2.

Table 3.2. Summary of investigated research questions.

Identifier	Question	Downstream tasks
RQ1	<i>How does the use of various pre-trained encoders, in combination with a query back mechanism, affect the quality of code change representations?</i>	—
RQ1.1	<i>How does the use of different pre-trained encoders affect the quality of code change representations in binary classification tasks?</i>	APCA, JIT-DP
RQ1.2	<i>How does the use of different pre-trained encoders affect the quality of code change representations in text generation tasks?</i>	CMG
RQ2	<i>How does applying a novel retrieval augmentation technique, instead of fine-tuning pre-trained encoders, impact the quality of code change representations?</i>	—
RQ2.1	<i>How does the number of feature vectors retrieved affect the quality of code change representations?</i>	APCA
RQ2.2	<i>How does the choice of retrieval method affect the quality of code change representations?</i>	APCA
RQ2.3	<i>How does the method of combining retrieved feature vectors affect the quality of code change representations?</i>	APCA, JIT-DP

3.5. Results and analysis

This section presents results that address the formulated research questions. For questions RQ1.1, RQ1.2, and RQ2.3, the results are reported as metric scores obtained on specific datasets. In contrast, for RQ2.1 and RQ2.2, the results are shown as averaged metric scores across multiple tested parameter configurations related to the examined variable. In addition, to provide a comprehensive understanding of the performance variability, boxplots are included to visualize the distribution of MCC and AUC across all combinations tested in these two RQs.

Following previous research [3, 33], the APCA task was evaluated using a 5-fold cross-validation, as adopted in these studies. For the remaining downstream tasks, predefined dataset splits into training, validation, and test sets were used, according to the original experimental setups reported in the previous work. This approach was taken to maintain experimental consistency and allow for a more accurate comparison of the proposed methods with existing baselines.

3.5.1. Impact of various encoders on query back mechanism

To explore the impact of using different pre-trained encoders in conjunction with the query back mechanism on CCR, this research formulates two sub-questions under the primary research question RQ1. The responses to RQ1.1 and RQ1.2 are presented below, followed by a general discussion summarizing the findings relevant to the overall RQ1.

RQ1.1: *How does the use of different pre-trained encoders affect the quality of code change representations in binary classification tasks?*

The evaluation metrics corresponding to RQ1.1 are presented for the APCA task in the CACHE-Small dataset in Table 3.3. For the JIT-DP task, results are reported on the Gerrit dataset in Table 3.4, on the Go dataset in Table 3.5, and on the JDT dataset in Table 3.6.

For the APCA task in CACHE-Small (Table 3.3), the CodeT5 encoder achieved the highest scores in terms of precision, F1 score, AUC, and MCC, outperforming all other encoders. However, the improvement over the baseline CodeBERT (used in the original CCR implementation) did not exceed 0.02 in any of these metrics. CodeT5+ with 220M parameters achieved the highest precision score, showing a marginal lead of almost 3 percentage points over CodeBERT, the most significant gap observed in this setup. Graphcodebert produced the highest recall, while CodeT5+ with 770M parameters matched CodeT5 in AUC performance. These results indicate that although there are differences between the models, they remain small and no encoder demonstrated a consistently significant advantage over the baseline.

In the case of the JIT-DP task in the Gerrit dataset (Table 3.4), it is evident that AUC alone – as used in previous work – provides an incomplete or even misleading evaluation.

Table 3.3. Comparison of encoder performance for RQ1.1 on the APCA task, evaluated on the CACHE-Small dataset.

Encoder	Accuracy	Precision	Recall	F1 score	AUC	MCC
CodeBERT (CCRep)	.822	.762	.884	.818	.888	.652
Graphcodebert	.809	.739	.898	.810	.895	.636
UniXcoder	.801	.740	.870	.798	.882	.616
CodeT5	.834	.792	.862	.825	.905	.671
CodeT5+ 220M	.819	.793	.815	.803	.894	.638
CodeT5+ 770M	.826	.781	.858	.816	.905	.657

Table 3.4. Comparison of encoder performance for RQ1.1 on the JIT-DP task, evaluated on the Gerrit dataset.

Encoder	Accuracy	Precision	Recall	F1 score	AUC	MCC
CodeBERT (CCRep)	.933	.118	.011	.020	.766	.017
Graphcodebert	.933	.118	.011	.020	.776	.017
UniXcoder	.933	.133	.011	.020	.760	.021
CodeT5	.923	.235	.101	.141	.748	.118
CodeT5+ 220M	.930	.216	.043	.071	.773	.071
CodeT5+ 770M	.931	.160	.021	.038	.758	.037

Despite appearing competitive in terms of AUC, most encoders performed poorly when evaluated using MCC, a more balanced metric that accounts for the class distribution. Only CodeT5 achieved an MCC value greater than zero in the first decimal place and also outperformed others in terms of precision, recall, and F1 score. All encoders showed very high accuracy, which can be attributed to the significant class imbalance in the dataset. CodeT5 demonstrated a slight but consistent advantage over other encoders, although overall model performance remained low, aside from accuracy and AUC.

On the Go dataset (Table 3.5), results again indicate that all encoders achieved similar performance levels. UniXcoder performed best in terms of accuracy and precision, while CodeT5 produced the highest recall and F1 score. CodeT5+ (220M) achieved the best AUC and MCC scores. In particular, the best-performing encoder in terms of precision and recall exceeded CodeBERT by at least 5.5 percentage points. However, while measurable, these gains were not substantial enough to suggest a significant improvement over baseline.

For the JDT dataset (Table 3.6), used in the JIT-DP task, CodeBERT, the default encoder in CCRep, performed best in almost all metrics. The only exception was precision,

Table 3.5. Comparison of encoder performance for RQ1.1 on the JIT-DP task, evaluated on the Go dataset.

Encoder	Accuracy	Precision	Recall	F1 score	AUC	MCC
CodeBERT (CCRep)	.701	.611	.623	.617	.756	.373
Graphcodebert	.702	.612	.623	.618	.760	.374
UniXcoder	.712	.668	.503	.574	.758	.371
CodeT5	.684	.577	.683	.625	.752	.359
CodeT5+ 220M	.710	.633	.592	.612	.760	.381
CodeT5+ 770M	.709	.636	.573	.603	.759	.375

Table 3.6. Comparison of encoder performance for RQ1.1 on the JIT-DP task, evaluated on the JDT dataset.

Encoder	Accuracy	Precision	Recall	F1 score	AUC	MCC
CodeBERT (CCRep)	.642	.657	.561	.605	.689	.284
Graphcodebert	.598	.667	.355	.463	.687	.211
UniXcoder	.596	.715	.290	.412	.679	.225
CodeT5	.604	.606	.545	.574	.667	.206
CodeT5+ 220M	.599	.675	.349	.460	.685	.216
CodeT5+ 770M	.610	.653	.433	.521	.660	.226

where CodeT5+ (220M) achieved the highest score. Once again, the differences among the encoders were relatively small, and no model showed a statistically significant advantage.

Overall, the results for RQ1.1 suggest that while some encoders, particularly CodeT5 and CodeT5+, occasionally outperform CodeBERT in specific metrics, the improvements are marginal. Across all datasets and tasks, the choice of encoder has a limited impact on model performance when used in combination with the query back mechanism. This observation is particularly important when considering trade-offs between model complexity, resource consumption, and performance gain.

RQ1.2: *How does the use of different pre-trained encoders affect the quality of code change representations in text generation tasks?*

The evaluation results related to RQ1.2 are presented in Table 3.7. These results were obtained by evaluating encoder performance in the CMG task using the FIRA dataset.

Among the encoders tested, the default encoder used in the CCRep architecture, CodeBERT, achieved the highest scores on the BLEU and METEOR metrics. On the

Table 3.7. Comparison of encoder performance for RQ1.2 on the CMG task, evaluated on the FIRA dataset.

Encoder	BLEU	METEOR	ROUGE-1	ROUGE-2	ROUGE-L
CodeBERT (CCRep)	19.694	15.841	23.858	6.927	23.408
CodeT5+	19.585	15.541	23.928	6.959	23.453
UniXcoder	18.763	14.710	22.808	6.668	22.436

other hand, CodeT5+ obtained the best performance in the ROUGE family of metrics, particularly ROUGE-1, ROUGE-2, and ROUGE-L. Despite these variations, the overall differences in the metric values between all encoders remain relatively small. No model consistently outperformed the others on all evaluation criteria, suggesting that encoder choice had only a minor impact on the performance of the CMG task in this experimental setting.

RQ1: *How does the use of various pre-trained encoders, in combination with a query back mechanism, affect the quality of code change representations?*

Based on the conducted experiments and evaluation across multiple tasks and datasets, there is no conclusive evidence that any specific pre-trained encoder significantly outperforms the others in terms of overall quality of code change representations. Although some encoders achieved marginally better results for particular metrics or datasets, these improvements were generally modest and not statistically significant. This observation holds despite the considerable variation in model size and architecture, which typically introduces challenges in fine-tuning and performance consistency.

These findings suggest that, while encoder selection does influence performance to some extent, the effect may be secondary compared to other architectural or task-specific factors, such as the design of the downstream model, the characteristics of the dataset or the integration of query back mechanisms.

3.5.2. Performance of the proposed retrieval augmentation approach

To evaluate the effectiveness of the proposed approach to improve CCR using RA methods, RQ2 was formulated. This research question is further divided into three sub-questions: RQ2.1, RQ2.2, and RQ2.3. The answers to these sub-questions are presented below, followed by a general discussion summarizing the key findings related to RQ2.

RQ2.1: *How does the number of feature vectors retrieved affect the quality of code change representations?*

Table 3.8. Performance comparison for different numbers of retrieved feature vectors (Top-k) for RQ2.1 on the APCA task, evaluated on the CACHE-Small dataset.

Top k retrieved	Accuracy	Precision	Recall	F1	AUC	MCC
3	.716	.720	.613	.660	.769	.424
5	.715	.722	.607	.657	.767	.423
7	.715	.722	.606	.657	.767	.423
10	.715	.722	.606	.657	.768	.422

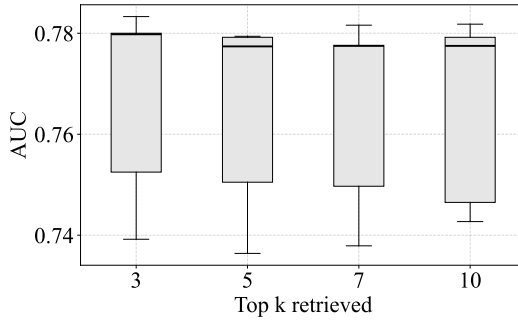


Fig. 3.1. Boxplot of AUC for different numbers of retrieved feature vectors (Top-k) for RQ2.1 on the APCA task, evaluated on the CACHE-Small dataset.

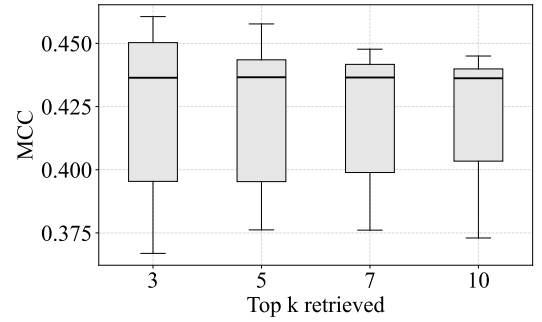


Fig. 3.2. Boxplot of MCC for different numbers of retrieved feature vectors (Top-k) for RQ2.1 on the APCA task, evaluated on the CACHE-Small dataset.

To investigate this, multiple combinations of retrieval and combination methods were tested for different values of top- k retrieved feature vectors. The average results for all methods are presented in Table 3.8. Furthermore, Figure 3.1 and Figure 3.2 show the distributions of AUC and MCC scores, respectively, using box plots.

The results indicate that the average performance across all evaluation metrics remains nearly identical regardless of the number of vectors retrieved. This consistency is also reflected in the boxplots, where the distributions are highly similar for different values of k . A minor decrease in variance for the MCC metric can be observed with larger values of k , suggesting a slightly more stable performance. However, this effect is negligible in practice. In general, these findings suggest that the specific number of recovered feature vectors has little or no impact on the effectiveness of CCR in the APCA task when evaluated in the CACHE-Small dataset.

RQ2.2: *How does the choice of retrieval method affect the quality of code change representations?*

This sub-question examines whether different distance metrics used for retrieving similar code change vectors influence the downstream performance. The average results for the cosine, Euclidean, and Manhattan retrieval methods – across all values of k and the

Table 3.9. Performance comparison for different retrieval methods of feature vectors for RQ2.2 on the APCA task, evaluated on the CACHE-Small dataset.

Retrieval method	Accuracy	Precision	Recall	F1	AUC	MCC
cosine	.717	.721	.657	.660	.768	.422
euclidean	.716	.613	.661	.657	.768	.426
manhattan	.714	.603	.655	.657	.768	.422

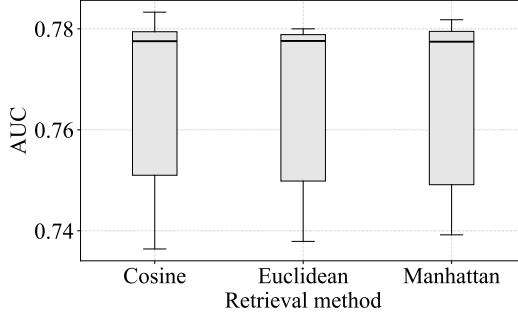


Fig. 3.3. Boxplot of AUC for different retrieval methods of feature vectors for RQ2.2 on the APCA task, evaluated on the CACHE-Small dataset.

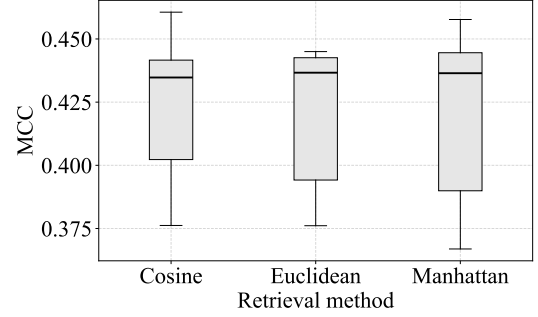


Fig. 3.4. Boxplot of MCC for different retrieval methods of feature vectors for RQ2.2 on the APCA task, evaluated on the CACHE-Small dataset.

combination methods – are shown in Table 3.9. The corresponding distributions of AUC and MCC are visualized in Figure 3.3 and Figure 3.4.

As observed in the results, the average metric values across the tested retrieval methods are nearly identical. The box plots confirm this by showing minimal differences in the distributions of AUC and MCC scores. Although the Euclidean method slightly outperforms the others in terms of MCC, the improvement is marginal and not statistically significant. These results indicate that, for the given task and dataset, the choice of retrieval method does not substantially affect the performance of the CCR model.

RQ2.3: *How does the method of combining retrieved feature vectors affect the quality of code change representations?*

To investigate how different methods for combining retrieved feature vectors impact the quality of CCRs, three variants of the proposed RA models were evaluated: RA-avg, RA-weight, and RA-att. These models utilize different strategies to aggregate the retrieved vectors and were tested using the cosine similarity retrieval method with $k = 5$. The evaluation was carried out on multiple datasets and tasks.

Table 3.10 presents the results for the APCA task in the CACHE-Small dataset. For the JIT-DP task, evaluations were performed on the Gerrit (Table 3.11), Go (Table 3.12), and JDT (Table 3.13) datasets.

As baselines, two models were selected: CCRep, a state-of-the-art approach for code

change representation, and the raw pre-trained CodeT5+ encoder without fine-tuning, which is also the foundation of the proposed method. Including the unmodified CodeT5 + allows a direct assessment of how much performance gain is achieved through the proposed RA approach.

Table 3.10. Performance comparison for different proposed models for RQ2.3 on the APCA task, evaluated on the CACHE-Small dataset.

Model	Accuracy	Precision	Recall	F1	AUC	MCC
CCRep	.822	.762	.884	.818	.888	.652
CodeT5+	.668	.686	.498	.575	.716	.326
RA-avg	.722	.719	.636	.674	.777	.437
RA-weight	.724	.725	.637	.675	.779	.442
RA-att	.691	.706	.556	.618	.736	.376

In the small CACHE-Small dataset, all proposed RA models outperformed the raw CodeT5+ encoder on all metrics, indicating that the retrieval component effectively enhances the quality of the representation. However, none of the proposed models were able to match the performance of CCRep, which achieved the highest scores in all metrics. This suggests that for small-scale datasets, the proposed approach may not be sufficient to compete with fine-tuned, specialized solutions such as CCRep.

Table 3.11. Performance comparison for different proposed models for RQ2.3 on the JIT-DP task, evaluated on the Gerrit dataset.

Model	Accuracy	Precision	Recall	F1	AUC	MCC
CCRep	.933	.118	.011	.020	.766	.017
CodeT5+	.928	.158	.032	.053	.723	.044
RA-avg	.922	.174	.064	.094	.771	.070
RA-weight	.935	.125	.005	.010	.761	.013
RA-att	.936	.163	.044	.069	.759	.037

In the case of the JIT-DP task on the large but highly imbalanced Gerrit dataset, the RA-avg model achieved the best overall results in most metrics, including precision, recall, F1 score, AUC, and MCC, except for accuracy, where RA-att slightly outperformed the others. However, despite these relative improvements, all models, including the state-of-the-art CCRep, struggled to provide satisfactory results, underlining the inherent difficulty of this dataset. In particular, the raw CodeT5+ encoder performed comparably well in some metrics, which emphasizes the limitations of both the fine-tuned and RA approaches in highly skewed datasets.

Table 3.12. Performance comparison for different proposed models for RQ2.3 on the JIT-DP task, evaluated on the Go dataset.

Model	Accuracy	Precision	Recall	F1	AUC	MCC
CCRep	.701	.611	.623	.617	.756	.376
CodeT5+	.683	.589	.593	.591	.747	.363
RA-avg	.700	.616	.612	.614	.755	.375
RA-weight	.701	.644	.609	.626	.758	.370
RA-att	.696	.634	.582	.607	.757	.370

In the largest data set, Go, the RA-weight model produced results comparable to those of CCRep. Both models achieved identical accuracy. RA-weight surpassed CCRep in precision, F1 score, and AUC, while CCRep retained a slight edge in recall and MCC. These findings suggest that the proposed RA approach is a competitive alternative to fine-tuned state-of-the-art models when applied to large-scale datasets. Additionally, the fact that no fine-tuning is required makes the approach significantly more computationally efficient.

Table 3.13. Performance comparison for different proposed models for RQ2.3 on the JIT-DP task, evaluated on the JDT dataset.

Model	Accuracy	Precision	Recall	F1	AUC	MCC
CCRep	.642	.657	.561	.605	.689	.284
CodeT5+	.628	.640	.548	.590	.657	.256
RA-avg	.633	.641	.567	.602	.667	.265
RA-weight	.619	.631	.533	.578	.675	.238
RA-att	.619	.643	.498	.561	.663	.240

On the relatively small JDT dataset, the results were similar to those observed for CACHE-Small. Although the proposed models outperformed the baseline raw encoder, they did not reach the performance of CCRep. Among the proposed methods, RA-avg achieved the highest recall, slightly exceeding CCRep in this metric, but the overall superiority of CCRep in most metrics remained evident. This again highlights that the benefits of retrieval-augmented representations may be limited on small-scale datasets.

RQ2: *How does applying a novel retrieval augmentation technique, instead of fine-tuning pre-trained encoders, impact the quality of code change representations?*

Based on the conducted experiments, the proposed, novel, RA-based approach demonstrates clear potential as an alternative to fine-tuning pre-trained encoders. Although its

effectiveness on smaller datasets is limited, failing to surpass state-of-the-art models such as CCRRep, it consistently improves upon the raw encoder baseline and achieves comparable performance on large datasets.

The key advantage of this method lies in its efficiency. By avoiding the computational cost of fine-tuning, the approach is more scalable and practical in scenarios with constrained resources or frequent model retraining needs. In summary, retrieval-augmented representation learning offers a promising trade-off between performance and efficiency, especially in large-scale applications.

3.5.3. Key findings from the empirical evaluation

The most important conclusions drawn from the empirical evaluation conducted for the research questions are as follows.

RQ1. The evaluation of various pre-trained code encoders, used in combination with the state-of-the-art query back mechanism, revealed that the choice of encoder did not significantly impact the overall performance of the CCR learning task. None of the tested encoders consistently outperformed the others across different datasets or tasks, indicating that the query-back mechanism itself is largely encoder-agnostic. This suggests that the effectiveness of this mechanism does not heavily rely on the specific architecture or pre-training objective of the encoder used.

RQ2. The novel RA technique proposed in this thesis, applied as an alternative to computationally expensive fine-tuning, demonstrated promising results. Specifically, it achieved performance comparable to that of the current state-of-the-art model, CCRRep, when evaluated on large datasets. This is particularly noteworthy given that the proposed method relies solely on pre-trained encoders without any task-specific fine-tuning. On smaller datasets, although the proposed approach did not surpass CCRRep, it still yielded clear improvements over the raw pre-trained encoder (CodeT5+), which was also used as a component of the RA architecture. These results suggest that RA can enhance CCRs in scenarios where fine-tuning is impractical or resource-constrained, while still maintaining a competitive level of performance.

4. Discussion and further work

This chapter presents general observations and critical reflections that emerged during the course of this study. It includes considerations regarding the tested variants of the proposed approach, as well as its current limitations and potential weaknesses. In addition, it outlines the threats to validity associated with both the systematic review of the literature and the empirical evaluation of the proposed method. Finally, recommendations for further work directions are discussed.

4.1. Variants of the proposed approach

During the investigation of different configurations of the proposed retrieval-augmented CCR model, several key insights were obtained. Specifically, experiments related to RQ2.1 and RQ2.2 assessed the impact of varying the number of feature vectors retrieved from the knowledge source and the techniques used to retrieve them, respectively. The results did not reveal substantial differences in the performance of the CCRs generated across these variants. This suggests that the number of vectors retrieved or the retrieval strategy alone may not play a dominant role in determining the effectiveness of the model. Instead, as investigated in RQ2.3, it appears that the method used to combine or integrate knowledge from the retrieved feature vectors has a more pronounced influence on the overall quality of the representations.

However, it was also observed that the performance of the proposed approach is significantly reduced in very small datasets, such as those used in the experiments for RQ2.1 and RQ2.2. This sensitivity to dataset size highlights a potential limitation of the method and calls for a cautious interpretation of the findings in such contexts. Due to computational constraints, experiments with substantially larger datasets were not conducted within the scope of this thesis. This remains an open area for further investigation.

4.2. Limitations

Although the RA approach proposed in this thesis represents a novel and previously unexplored direction in the field of CCR, it is not without limitations.

The first and most prominent limitation, as discussed in the previous section, is the reduced effectiveness of the method when applied to small data sets. Although it offers a slight improvement over using a raw, pre-trained encoder without fine-tuning,

the gain is minimal and insufficient to compete with state-of-the-art methods that have been specifically optimized for such scenarios. This outcome is understandable given the underlying assumption of the approach: its strength lies in leveraging external knowledge through retrieval, which becomes less impactful when data availability is low or when the retrieved context is not sufficiently diverse.

Another limitation comes from the model compatibility requirements. In theory, the proposed solution can be integrated with any PTM that includes an encoder component. However, for effective performance, the chosen PTM must be pre-trained on code-related data. Applying a model that lacks prior exposure to programming languages or code semantics would result in significantly weaker performance, especially when compared to a fine-tuned variant of such a model. Therefore, while the method avoids the computational cost of fine-tuning, it still depends heavily on the availability and quality of specialized pre-trained encoders.

Moreover, there is a growing trend in the LLM community toward developing decoder-only architectures for large-scale models. These models, such as GPT-style transformers, cannot be directly utilized within the RA setup proposed in this work, as they lack an encoder module required for generating intermediate vector representations. This trend excludes many of the most powerful and widely used PTMs from being employed in this framework.

A similar constraint applies to encoder-decoder architectures that follow a "shallow encoder and deep decoder" design paradigm [66]. In such cases, the encoder component may be too weak to produce high-quality embeddings, thereby limiting the effectiveness of the RA approach. This narrows the selection of suitable models and may hinder the applicability of the method as newer model architectures continue to evolve.

4.3. Threats to validity

Given that this thesis consists of both an SR and a proposal of a technical solution, the potential threats to validity are discussed separately for each component.

4.3.1. Validity of the systematic literature review

Internal validity: One of the primary threats to internal validity in the context of SR is the fact that the review process was carried out by a single researcher. This may have introduced bias in the selection, inclusion, or interpretation of the reviewed literature. To mitigate this risk, the process followed state-of-the-art guidelines for conducting SRs as outlined by Kitchenham et al. [22, 24]. Furthermore, a formal review protocol was developed prior to conducting the SR. This protocol, provided in Appendix A, was reviewed and approved by the thesis supervisor. In addition, backward and forward snowballing

techniques were employed to reduce the likelihood of omitting relevant studies, especially those not initially returned by the selected databases.

External validity: The external validity of the review may be limited by the restriction to English-language publications. Although the vast majority of relevant work in software engineering and machine learning is published in English, it is possible that some valuable contributions were missed in other languages. Another potential threat relates to the choice of indexing services, namely Scopus for the initial query and Google Scholar for forward snowballing. Although these platforms are widely recognized and comprehensive, they may still overlook some domain-specific publications. However, the risk posed by these limitations is considered minimal.

4.3.2. Validity of the Solution and Its Evaluation

Construct validity: A notable threat to construct validity arises from the evaluation setup used in different tasks. Although cross-validation was applied in the APCA task, the JIT-DP and CMG tasks relied on predefined train/validation/test splits from prior works. This choice was made to ensure a fair and direct comparison with the existing baseline methods. To mitigate randomness and increase the reliability of the results, all models were executed multiple times and average scores were reported. Furthermore, to reduce the risk of misleading conclusions due to narrow evaluation criteria, a broader set of evaluation metrics was used, particularly in the JIT-DP task, where previous studies had used only limited performance indicators.

Internal validity: Following previous work, input sequences larger than 512 tokens were trimmed to comply with the maximum token limit imposed by the PTMs employed. This truncation can lead to information loss and degrade model performance. To mitigate this issue in the proposed RA solution, the input was split into three separate components: added code, removed code, and commit message, effectively maximizing the available context window. Furthermore, due to constraints on computational resources and time, hyperparameter tuning in the first module was limited. To compensate, the recommended default values provided by the authors of the respective encoders were used, which are reported to perform well in fine-tuning scenarios. Another concern is the quality and consistency of the training data, which can significantly affect the performance of the model. To address this, the same datasets used in the baseline experiments were also used in the evaluation of the proposed method. Finally, the correctness of the custom implementation is a potential threat. To minimize implementation-related errors, standard software engineering practices were applied, including static code analysis using linters and type checkers, as well as a thorough manual review of the implementation.

External validity: The main concern with external validity is the limited scope of the tasks and datasets evaluated. The first module was tested on three tasks and the

second (proposed) module on two. Given the wide range of downstream tasks that rely on code-change representations, the generalizability of the findings may be questioned. It is possible that the proposed approach may not perform as effectively on tasks or datasets not considered in this thesis. However, this threat is partially mitigated by the diversity of tasks included—each varying in their input structure and output objectives. This diversity provides a reasonable, though not exhaustive, indication of the potential of the method in different contexts.

4.4. Further work

During the development and evaluation of the proposed novel approach, several potential directions for improvement and future research were identified.

One of the most apparent areas for enhancement is the preprocessing stage. In the current implementation, the input data is divided into three distinct components: added code, removed code, and the commit message. While this design facilitates a structured representation of code changes, it fails to explicitly model the relationships between added and removed code segments, or to capture the surrounding code context in which these changes occur. This limitation could potentially affect the quality of learned representations, particularly for changes that depend on subtle contextual or semantic cues. Future work may explore more context-aware preprocessing techniques, such as including unchanged code surrounding, generating edit scripts, or employing diff-aware encoding methods that maintain alignment between code additions and deletions.

Another key place for improvement lies in the input length constraint imposed by the employed pre-trained transformer models. As most encoder-based models are restricted to a maximum of 512 tokens, parts of the input are occasionally truncated, which can result in the loss of critical information, especially for larger commits. Addressing this issue may involve experimenting with hierarchical encoding approaches, input fragmentation strategies, or utilizing models with extended input capacities.

Furthermore, the experiments conducted to answer the research questions RQ2.1 and RQ2.2 were performed on relatively small datasets. As a result, the generalizability and reliability of the observed outcomes may be limited. Future research should aim to replicate these experiments on larger and more diverse datasets to validate trends and confirm the robustness of the findings.

In addition, the mechanism used to combine retrieved knowledge vectors from external sources – identified as a crucial factor in overall performance (as discussed in RQ2.3). Investigating more advanced combining techniques may further enhance the quality of the resulting code change representations.

Conclusion

This thesis focused on two main areas: a systematic review of the literature (SR) of existing research in the field of code change representations (CCRs), and the design and evaluation of a novel solution aimed at learning representations of code changes via artificial intelligence code model.

As part of the SR, a snowballing methodology was applied to identify a total of 95 unique and relevant research papers from an initial pool of over 18,000 publications (including duplicates). These studies represent a wide variety of approaches to the problem of CCR, which can be broadly categorized into three main groups: methods based on lexical representations of code (typically through tokenization), tree-based representations (most notably using Abstract Syntax Trees), and graph-based representations that aim to capture more complex semantic structures. This diversity reflects the multidimensional nature of the code and the corresponding challenges in modeling its changes effectively.

The review also revealed a clear trend in recent years: the increasing adoption of pre-trained models (PTMs) for representing code changes. These models have demonstrated significant improvements in the capture of the semantics of code modifications compared to earlier hand-engineered or rule-based approaches. However, the review also highlighted a notable gap in the field, the limited availability of general-purpose CCR models that are task-agnostic and can be effectively reused across a variety of downstream tasks. Most existing solutions are tightly coupled to specific tasks such as defect prediction or commit message generation.

To address this gap, the second part of the thesis focused on the implementation and evaluation of methods for learning general-purpose CCRs. This phase of the work was divided into two modules. The first module involved an empirical investigation of a state-of-the-art CCR method, CCRRep [3]. This module examined how the choice of pre-trained encoder affects the quality of the learned code change representations, particularly when used in combination with the query-back mechanism proposed in the original CCRRep architecture. To facilitate a more comprehensive and reliable comparison with this method, a broader set of evaluation metrics was collected than in previous studies. The results indicated that, despite the considerable architectural and training differences between the encoders, the overall performance of the system remained relatively stable. This suggests that the quality of CCRs in such architectures may not be strongly influenced by the encoder alone.

The second module of the thesis introduced a novel solution based on retrieval

augmentation (RA), an approach that was not previously explored within the CCR domain. This method was designed to serve as a lightweight alternative to expensive fine-tuning of large-scale language models. Instead of adapting a model to a specific task via fine-tuning, it leverages pre-trained encoders and enriches their representations using contextually similar code changes retrieved from an external index. Experimental results demonstrated that this approach achieved performance comparable to that of CCRep when evaluated on large datasets, despite relying solely on frozen (non fine-tuned) encoders. This is a particularly promising finding in scenarios where fine-tuning is infeasible due to limited computational resources. On smaller datasets, although the proposed approach did not outperform CCRep, it still showed measurable improvements over the raw pre-trained encoder, which served as one of the main components in the RA approach.

In general, the results of this thesis indicate that the RA-based method is a viable and efficient strategy to improve CCR. It offers a promising path forward for building adaptable and scalable CCR systems, especially in environments where task-specific fine-tuning is not practical. Future research can expand upon this foundation by refining the retrieval and aggregation strategies and evaluating the approach on a broader range of tasks and programming languages.

Acknowledgements

Taking advantage of this modest space, I would like to express my gratitude to everyone who contributed directly or indirectly to the successful completion of this Master's thesis.

First, my sincere thanks go to my thesis supervisor, Professor Lech Madeyski, for his invaluable support throughout this project, for the knowledge he generously shared during my master's studies, and for providing access to the computing resources used in part of this research.

I would also like to thank my friends and family for their unwavering support (and patience). Special thanks go to Tomek and Przemek, with whom I have built several projects and engaged in endless discussions, often about seemingly trivial things that somehow always turned out to be unexpectedly enlightening.

Finally, I extend my appreciation to all the instructors and lecturers I met during my studies, whether for the knowledge they imparted or the psychological resilience that their courses helped cultivate. Both, as it turns out, are quite useful in the real world.

Bibliography

- [1] I. I. Brudaru, A. Zeller, What is the long-term impact of changes?, in: Proceedings of the 2008 International Workshop on Recommendation Systems for Software Engineering, RSSE '08, Association for Computing Machinery, New York, NY, USA, 2008, p. 30–32. URL: <https://doi.org/10.1145/1454247.1454257>. doi:10.1145/1454247.1454257.
- [2] T. Hoang, H. J. Kang, D. Lo, J. Lawall, Cc2vec: distributed representations of code changes, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, ICSE '20, Association for Computing Machinery, New York, NY, USA, 2020, p. 518–529. URL: <https://doi.org/10.1145/3377811.3380361>. doi:10.1145/3377811.3380361.
- [3] Z. Liu, Z. Tang, X. Xia, X. Yang, Ccrep: Learning code change representations via pre-trained code model and query back, in: Proceedings of the 45th International Conference on Software Engineering, ICSE '23, IEEE Press, 2023, p. 17–29. URL: <https://doi.org/10.1109/ICSE48619.2023.00014>. doi:10.1109/ICSE48619.2023.00014.
- [4] Z. Li, S. Lu, D. Guo, N. Duan, S. Jannu, G. Jenks, D. Majumder, J. Green, A. Svyatkovskiy, S. Fu, N. Sundaresan, Automating code review activities by large-scale pre-training, in: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Association for Computing Machinery, New York, NY, USA, 2022, p. 1035–1047. URL: <https://doi.org/10.1145/3540250.3549081>. doi:10.1145/3540250.3549081.
- [5] H. Tian, K. Liu, A. K. Kaboré, A. Koyuncu, L. Li, J. Klein, T. F. Bissyandé, Evaluating representation learning of code changes for predicting patch correctness in program repair, in: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering, ASE '20, Association for Computing Machinery, New York, NY, USA, 2021, p. 981–992. URL: <https://doi.org/10.1145/3324884.3416532>. doi:10.1145/3324884.3416532.
- [6] T. Hoang, H. Khanh Dam, Y. Kamei, D. Lo, N. Ubayashi, Deepjit: An end-to-end deep learning framework for just-in-time defect prediction, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 34–45. doi:10.1109/MSR.2019.00016.
- [7] S. Jiang, A. Armaly, C. McMillan, Automatically generating commit messages from

- diffs using neural machine translation, in: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), 2017, pp. 135–146. doi:10.1109/ASE.2017.8115626.
- [8] S. Kim, E. J. Whitehead, Y. Zhang, Classifying software changes: Clean or buggy?, *IEEE Transactions on Software Engineering* 34 (2008) 181–196. doi:10.1109/TSE.2007.70773.
- [9] Y. Kamei, T. Fukushima, S. McIntosh, K. Yamashita, N. Ubayashi, A. E. Hassan, Studying just-in-time defect prediction using cross-project models, *Empirical Softw. Engg.* 21 (2016) 2072–2106. URL: <https://doi.org/10.1007/s10664-015-9400-x>. doi:10.1007/s10664-015-9400-x.
- [10] H. Ye, J. Gu, M. Martinez, T. Durieux, M. Monperrus, Automated classification of overfitting patches with statically extracted code features, *IEEE Transactions on Software Engineering* 48 (2022) 2920–2938. doi:10.1109/TSE.2021.3071750.
- [11] W. D. Mellin, Work with new electronic ‘brains’ opens field for army math experts, *Hammond Times* 10 (1957).
- [12] H. P. Samoaa, F. Bayram, P. Salza, P. Leitner, A systematic mapping study of source code representation for deep learning in software engineering, *IET Software* 16 (2022) 351–385. URL: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/sfw2.12064>. doi:<https://doi.org/10.1049/sfw2.12064>.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, Curran Associates Inc., Red Hook, NY, USA, 2017, p. 6000–6010.
- [14] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, CodeBERT: A pre-trained model for programming and natural languages, in: T. Cohn, Y. He, Y. Liu (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, Association for Computational Linguistics, Online, 2020, pp. 1536–1547. URL: <https://aclanthology.org/2020.findings-emnlp.139/>. doi:10.18653/v1/2020.findings-emnlp.139.
- [15] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, M. Zhou, Graphcodebert: Pre-training code representations with data flow, 2021. URL: <https://arxiv.org/abs/2009.08366>. arXiv:2009.08366.
- [16] Y. Wang, W. Wang, S. Joty, S. C. Hoi, CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: M.-F. Moens, X. Huang, L. Specia, S. W.-t. Yih (Eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021, pp. 8696–8708. URL: <https://aclanthology.org/2021.emnlp-main.685/>.

doi:10.18653/v1/2021.emnlp-main.685.

- [17] C. Niu, C. Li, V. Ng, J. Ge, L. Huang, B. Luo, SPT-Code: Sequence-to-Sequence Pre-Training for Learning Source Code Representations, in: 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE), IEEE Computer Society, Los Alamitos, CA, USA, 2022, pp. 1–13. URL: <https://doi.ieeecomputersociety.org/10.1145/3510003.3510096>. doi:10.1145/3510003.3510096.
- [18] P. Yin, G. Neubig, M. Allamanis, M. Brockschmidt, A. L. Gaunt, Learning to represent edits, 2019. URL: <https://openreview.net/forum?id=BJl6AjC5F7>.
- [19] B. Lin, S. Wang, M. Wen, X. Mao, Context-aware code change embedding for better patch correctness assessment, ACM Trans. Softw. Eng. Methodol. 31 (2022). URL: <https://doi.org/10.1145/3505247>. doi:10.1145/3505247.
- [20] R. Dyer, H. A. Nguyen, H. Rajan, T. N. Nguyen, Boa: A language and infrastructure for analyzing ultra-large-scale software repositories, in: 2013 35th International Conference on Software Engineering (ICSE), 2013, pp. 422–431. doi:10.1109/ICSE.2013.6606588.
- [21] Z. Liu, X. Xia, A. E. Hassan, D. Lo, Z. Xing, X. Wang, Neural-machine-translation-based commit message generation: how far are we?, in: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, ASE '18, Association for Computing Machinery, New York, NY, USA, 2018, p. 373–384. URL: <https://doi.org/10.1145/3238147.3238190>. doi:10.1145/3238147.3238190.
- [22] B. Kitchenham, S. Charters, D. Budgen, P. Brereton, M. Turner, S. Linkman, M. Jørgensen, E. Mendes, G. Visaggio, Guidelines for performing systematic literature reviews in software engineering, Technical Report, Keele University, 2007.
- [23] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, EASE '14, Association for Computing Machinery, New York, NY, USA, 2014, pp. 1–10. doi:10.1145/2601248.2601268.
- [24] B. Kitchenham, L. Madeyski, D. Budgen, Segress: Software engineering guidelines for reporting secondary studies, IEEE Transactions on Software Engineering 49 (2023) 1273–1298. doi:10.1109/TSE.2022.3174092.
- [25] Z. Xian, R. Huang, D. Towey, C. Fang, Z. Chen, Transformcode: A contrastive learning framework for code embedding via subtree transformation, IEEE Trans. Softw. Eng. 50 (2024) 1600–1619. URL: <https://doi.org/10.1109/TSE.2024.3393419>. doi:10.1109/TSE.2024.3393419.
- [26] U. Alon, M. Zilberstein, O. Levy, E. Yahav, code2vec: learning distributed representations of code, Proc. ACM Program. Lang. 3 (2019). URL: <https://doi.org/10.1145/3290353>. doi:10.1145/3290353.

- [27] J. Dong, Y. Lou, Q. Zhu, Z. Sun, Z. Li, W. Zhang, D. Hao, Fira: fine-grained graph-based code change representation for automated commit message generation, in: Proceedings of the 44th International Conference on Software Engineering, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 970–981. URL: <https://doi.org/10.1145/3510003.3510069>. doi:10.1145/3510003.3510069.
- [28] R. Cabrera Lozoya, A. Baumann, A. Sabetta, M. Bezzi, Commit2vec: Learning distributed representations of code changes, SN Comput. Sci. 2 (2021). URL: <https://doi.org/10.1007/s42979-021-00566-z>. doi:10.1007/s42979-021-00566-z.
- [29] X. Cheng, G. Zhang, H. Wang, Y. Sui, Path-sensitive code embedding via contrastive learning for software vulnerability detection, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022, Association for Computing Machinery, New York, NY, USA, 2022, p. 519–531. URL: <https://doi.org/10.1145/3533767.3534371>. doi:10.1145/3533767.3534371.
- [30] M. White, M. Tufano, C. Vendome, D. Poshyvanyk, Deep learning code fragments for code clone detection, in: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, ASE '16, Association for Computing Machinery, New York, NY, USA, 2016, p. 87–98. URL: <https://doi.org/10.1145/2970276.2970326>. doi:10.1145/2970276.2970326.
- [31] J. Li, P. He, J. Zhu, M. R. Lyu, Software defect prediction via convolutional neural network, in: 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS), 2017, pp. 318–328. doi:10.1109/QRS.2017.42.
- [32] N. Shiri Harzevili, A. Boaye Belle, J. Wang, S. Wang, Z. M. J. Jiang, N. Nagappan, A systematic literature review on automated software vulnerability detection using machine learning, ACM Comput. Surv. 57 (2024). URL: <https://doi.org/10.1145/3699711>. doi:10.1145/3699711.
- [33] W. Zou, Z. Shen, J. Ge, C. Li, B. Luo, Ccaf: Learning code change via adapterfusion, in: Proceedings of the 15th Asia-Pacific Symposium on Internetware, Internetware '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 219–228. URL: <https://doi.org/10.1145/3671016.3671399>. doi:10.1145/3671016.3671399.
- [34] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, I. Gurevych, Adapterfusion: Non-destructive task composition for transfer learning, 2021. URL: <https://arxiv.org/abs/2005.00247>. arXiv:2005.00247.
- [35] F. Zhang, B. Chen, Y. Zhao, X. Peng, Slice-based code change representation learning, in: 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), 2023, pp. 319–330. doi:10.1109/SANER56733.2023.

00038.

- [36] X. Zhou, B. Xu, D. Han, Z. Yang, J. He, D. Lo, CCBERT: Self-Supervised Code Change Representation Learning , in: 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 182–193. URL: <https://doi.ieeecomputersociety.org/10.1109/ICSME58846.2023.00028>. doi:10.1109/ICSME58846.2023.00028.
- [37] E. Shi, Y. Wang, L. Du, H. Zhang, S. Han, D. Zhang, H. Sun, Cocoast: Representing source code via hierarchical splitting and reconstruction of abstract syntax trees, Empirical Softw. Engg. 28 (2023). URL: <https://doi.org/10.1007/s10664-023-10378-9>. doi:10.1007/s10664-023-10378-9.
- [38] W. Ma, M. Zhao, E. Soremekun, Q. Hu, J. M. Zhang, M. Papadakis, M. Cordy, X. Xie, Y. L. Traon, Graphcode2vec: generic code embedding via lexical and program dependence analyses, in: Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 524–536. URL: <https://doi.org/10.1145/3524842.3528456>. doi:10.1145/3524842.3528456.
- [39] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, J. Yin, Unixcoder: Unified cross-modal pre-training for code representation, 2022. URL: <https://arxiv.org/abs/2203.03850>. arXiv:2203.03850.
- [40] A. R. Mandli, S. Rajput, T. Sharma, Comet: Generating commit messages using delta graph context representation, J. Syst. Softw. 222 (2025). URL: <https://doi.org/10.1016/j.jss.2024.112307>. doi:10.1016/j.jss.2024.112307.
- [41] S. Nguyen, T.-T. Nguyen, T. T. Vu, T.-D. Do, K.-T. Ngo, H. D. Vo, Code-centric learning-based just-in-time vulnerability detection, J. Syst. Softw. 214 (2024). URL: <https://doi.org/10.1016/j.jss.2024.112014>. doi:10.1016/j.jss.2024.112014.
- [42] Z. Zeng, Y. Zhang, H. Zhang, L. Zhang, Deep just-in-time defect prediction: how far are we?, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2021, Association for Computing Machinery, New York, NY, USA, 2021, p. 427–438. URL: <https://doi.org/10.1145/3460319.3464819>. doi:10.1145/3460319.3464819.
- [43] H. Wang, X. Xia, D. Lo, Q. He, X. Wang, J. Grundy, Context-aware retrieval-based deep commit message generation, ACM Trans. Softw. Eng. Methodol. 30 (2021). URL: <https://doi.org/10.1145/3464689>. doi:10.1145/3464689.
- [44] B. Casey, J. C. S. Santos, G. Perry, A survey of source code representations for machine learning-based cybersecurity tasks, ACM Comput. Surv. 57 (2025). URL: <https://doi.org/10.1145/3721977>. doi:10.1145/3721977.
- [45] T. Kudo, J. Richardson, Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing, arXiv preprint arXiv:1808.06226

- (2018).
- [46] K. W. Church, Word2vec, *Natural Language Engineering* 23 (2017) 155–162.
 - [47] Q. Le, T. Mikolov, Distributed representations of sentences and documents, in: *International conference on machine learning*, PMLR, 2014, pp. 1188–1196.
 - [48] E. Spirin, E. Bogomolov, V. Kovalenko, T. Bryksin, Psiminer: A tool for mining rich abstract syntax trees from code, in: *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, IEEE, 2021, pp. 13–17.
 - [49] D. G. Fritz, R. G. Sargent, An overview of hierarchical control flow graph models, in: *Proceedings of the 27th Conference on Winter Simulation, WSC '95*, IEEE Computer Society, USA, 1995, p. 1347–1355. URL: <https://doi.org/10.1145/224401.224819>. doi:10.1145/224401.224819.
 - [50] J. Ferrante, K. J. Ottenstein, J. D. Warren, The program dependence graph and its use in optimization, *ACM Trans. Program. Lang. Syst.* 9 (1987) 319–349. URL: <https://doi.org/10.1145/24039.24041>. doi:10.1145/24039.24041.
 - [51] L. Madeyski, B. Kitchenham, N.-T. Nguyen, M. Núñez, B. Trawiński, Would wider adoption of reproducible research be beneficial for empirical software engineering research?, *J. Intell. Fuzzy Syst.* 32 (2017) 1509–1521. URL: <https://doi.org/10.3233/JIFS-169146>. doi:10.3233/JIFS-169146.
 - [52] M. Gardner, J. Grus, M. Neumann, O. Tafjord, P. Dasigi, N. F. Liu, M. Peters, M. Schmitz, L. Zettlemoyer, AllenNLP: A deep semantic natural language processing platform, in: E. L. Park, M. Hagiwara, D. Milajevs, L. Tan (Eds.), *Proceedings of Workshop for NLP Open Source Software (NLP-OSS)*, Association for Computational Linguistics, Melbourne, Australia, 2018, pp. 1–6. URL: <https://aclanthology.org/W18-2501/>. doi:10.18653/v1/W18-2501.
 - [53] G. M. Kurtzer, V. Sochat, M. W. Bauer, Singularity: Scientific containers for mobility of compute, *PLOS ONE* 12 (2017) 1–20. URL: <https://doi.org/10.1371/journal.pone.0177459>. doi:10.1371/journal.pone.0177459.
 - [54] A. Paszke, Pytorch: An imperative style, high-performance deep learning library, *arXiv preprint arXiv:1912.01703* (2019).
 - [55] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al., Huggingface’s transformers: State-of-the-art natural language processing, *arXiv preprint arXiv:1910.03771* (2019).
 - [56] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, *Nature* 585 (2020) 357–362. URL: <https://doi.org/10.1038/s41586-020-2649-2>. doi:10.1038/s41586-020-2649-2.

- [57] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, *Journal of Machine Learning Research* 12 (2011) 2825–2830.
- [58] J. D. Hunter, Matplotlib: A 2d graphics environment, *Computing in Science & Engineering* 9 (2007) 90–95. doi:10.1109/MCSE.2007.55.
- [59] R. Kohavi, A study of cross-validation and bootstrap for accuracy estimation and model selection, in: *Proceedings of the 14th International Joint Conference on Artificial Intelligence - Volume 2, IJCAI’95*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1995, p. 1137–1143.
- [60] Y. Yao, L. Rosasco, A. Caponnetto, On early stopping in gradient descent learning, *Constructive approximation* 26 (2007) 289–315. doi:10.1007/s00365-006-0663-2.
- [61] L. Prechelt, Early stopping-but when?, in: *Neural Networks: Tricks of the trade*, Springer, 2002, pp. 55–69. doi:10.1007/3-540-49430-8_3.
- [62] Q. Li, Y.-L. Chen, *Data Flow Diagram*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 85–97. URL: https://doi.org/10.1007/978-3-540-89556-5_4. doi:10.1007/978-3-540-89556-5_4.
- [63] J. Ni, G. Hernandez Abrego, N. Constant, J. Ma, K. Hall, D. Cer, Y. Yang, Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models, in: S. Muresan, P. Nakov, A. Villavicencio (Eds.), *Findings of the Association for Computational Linguistics: ACL 2022*, Association for Computational Linguistics, Dublin, Ireland, 2022, pp. 1864–1874. URL: <https://aclanthology.org/2022.findings-acl.146/>. doi:10.18653/v1/2022.findings-acl.146.
- [64] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, *J. Mach. Learn. Res.* 21 (2020).
- [65] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, S. C. Hoi, Codet5+: Open code large language models for code understanding and generation, *arXiv preprint arXiv:2305.07922* (2023).
- [66] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, T. Hubert, P. Choy, C. de Masson d’Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Goyal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, O. Vinyals, Competition-level code generation with alphacode, *Science* 378 (2022) 1092–1097. URL: <https://www.science.org/doi/abs/10.1126/science.abq1158>. doi:10.1126/science.abq1158. arXiv:<https://www.science.org/doi/pdf/10.1126/science.abq1158>.
- [67] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler,

- M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive nlp tasks, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, volume 33, Curran Associates, Inc., 2020, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
- [68] A. Long, W. Yin, T. Ajanthan, V. Nguyen, P. Purkait, R. Garg, A. Blair, C. Shen, A. van den Hengel, Retrieval augmented classification for long-tail visual recognition, in: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 6949–6959. doi:10.1109/CVPR52688.2022.00683.
- [69] S. Gur, N. Neverova, C. Stauffer, S.-N. Lim, D. Kiela, A. Reiter, Cross-modal retrieval augmentation for multi-modal classification, in: M.-F. Moens, X. Huang, L. Specia, S. W.-t. Yih (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2021*, Association for Computational Linguistics, Punta Cana, Dominican Republic, 2021, pp. 111–123. URL: <https://aclanthology.org/2021.findings-emnlp.11/>. doi:10.18653/v1/2021.findings-emnlp.11.
- [70] T. Abdullahi, R. Singh, C. Eickhoff, Retrieval augmented zero-shot text classification, in: *Proceedings of the 2024 ACM SIGIR International Conference on Theory of Information Retrieval, ICTIR '24*, Association for Computing Machinery, New York, NY, USA, 2024, p. 195–203. URL: <https://doi.org/10.1145/3664190.3672514>. doi:10.1145/3664190.3672514.
- [71] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using Siamese BERT-networks, in: K. Inui, J. Jiang, V. Ng, X. Wan (Eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Association for Computational Linguistics, Hong Kong, China, 2019, pp. 3982–3992. URL: <https://aclanthology.org/D19-1410/>. doi:10.18653/v1/D19-1410.
- [72] W. Zhong, H. Ge, H. Ai, C. Li, K. Liu, J. Ge, B. Luo, Standup4npr: Standardizing setup for empirically comparing neural program repair systems, in: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, ASE '22*, Association for Computing Machinery, New York, NY, USA, 2023. URL: <https://doi.org/10.1145/3551349.3556943>. doi:10.1145/3551349.3556943.
- [73] H. Ye, M. Martinez, T. Durieux, M. Monperrus, A comprehensive study of automatic program repair on the quixbugs benchmark, *Journal of Systems and Software* 171 (2021) 110825. URL: <https://www.sciencedirect.com/science/article/pii/S0164121220302193>. doi:https://doi.org/10.1016/j.jss.2020.110825.
- [74] R. Just, D. Jalali, M. D. Ernst, Defects4j: a database of existing faults to enable controlled testing studies for java programs, in: *Proceedings of the 2014 Inter-*

- national Symposium on Software Testing and Analysis, ISSTA 2014, Association for Computing Machinery, New York, NY, USA, 2014, p. 437–440. URL: <https://doi.org/10.1145/2610384.2628055>. doi:10.1145/2610384.2628055.
- [75] D. Chicco, G. Jurman, The matthews correlation coefficient (mcc) should replace the roc auc as the standard metric for assessing binary classification, *BioData Mining* 16 (2023) 4.
- [76] Z. Mahmood, D. Bowes, P. C. R. Lane, T. Hall, What is the impact of imbalance on software defect prediction performance?, in: *Proceedings of the 11th International Conference on Predictive Models and Data Analytics in Software Engineering, PROMISE '15*, Association for Computing Machinery, New York, NY, USA, 2015. URL: <https://doi.org/10.1145/2810146.2810150>. doi:10.1145/2810146.2810150.
- [77] C. X. Ling, J. Huang, H. Zhang, Auc: a better measure than accuracy in comparing learning algorithms, in: *Proceedings of the 16th Canadian Society for Computational Studies of Intelligence Conference on Advances in Artificial Intelligence, AI'03*, Springer-Verlag, Berlin, Heidelberg, 2003, p. 329–341.
- [78] W. Tao, Y. Wang, E. Shi, L. Du, S. Han, H. Zhang, D. Zhang, W. Zhang, On the Evaluation of Commit Message Generation Models: An Experimental Study, in: *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE Computer Society, Los Alamitos, CA, USA, 2021, pp. 126–136. URL: <https://doi.ieeecomputersociety.org/10.1109/ICSME52107.2021.00018>. doi:10.1109/ICSME52107.2021.00018.
- [79] A. Lavie, A. Agarwal, Meteor: an automatic metric for mt evaluation with high levels of correlation with human judgments, in: *Proceedings of the Second Workshop on Statistical Machine Translation, StatMT '07*, Association for Computational Linguistics, USA, 2007, p. 228–231.
- [80] C.-Y. Lin, ROUGE: A package for automatic evaluation of summaries, in: *Text Summarization Branches Out*, Association for Computational Linguistics, Barcelona, Spain, 2004, pp. 74–81. URL: <https://aclanthology.org/W04-1013/>.

List of Figures

1.1.	Steps of the snowballing procedure.	8
1.2.	The course of the literature review.	11
1.3.	Distribution of identified papers by publication year.	12
1.4.	Representation of the code snippet from Listing 1.2 using abstract syntax tree.	19
1.5.	Representation of the code snippet from Listing 1.2 using control flow graph.	20
1.6.	Representation of the code snippet from Listing 1.2 using program dependence graph.	21
2.1.	Package diagram of RAFramework.	25
2.2.	Class diagram of RAFramework.	26
2.3.	The overall framework of CCRep.	28
2.4.	Overview of the proposed RAframework.	29
2.5.	Visual representation of RA-avg as a combine layer.	33
2.6.	Visual representation of RA-weight as a combine layer.	33
2.7.	Visual representation of RA-att as a combine layer.	34
3.1.	Boxplot of AUC for different numbers of retrieved feature vectors (Top-k) for RQ2.1 on the APCA task, evaluated on the CACHE-Small dataset.	46
3.2.	Boxplot of MCC for different numbers of retrieved feature vectors (Top-k) for RQ2.1 on the APCA task, evaluated on the CACHE-Small dataset.	46
3.3.	Boxplot of AUC for different retrieval methods of feature vectors for RQ2.2 on the APCA task, evaluated on the CACHE-Small dataset.	47
3.4.	Boxplot of MCC for different retrieval methods of feature vectors for RQ2.2 on the APCA task, evaluated on the CACHE-Small dataset.	47

List of Listings

1.1	The Scopus query used to gather papers for the start set.	10
1.2	A sample Python code snippet.	18
1.3	A tokenized code snippet from Listing 1.2.	18

List of Tables

1.1.	Index terms in initial papers from supervisor.	9
1.2.	The number of papers identified and reviewed at each stage of the literature review.	12
1.3.	A brief summary of datasets.	16
3.1.	Summary of datasets used for downstream tasks.	36
3.2.	Summary of investigated research questions.	41
3.3.	Comparison of encoder performance for RQ1.1 on the APCA task, evaluated on the CACHE-Small dataset.	43
3.4.	Comparison of encoder performance for RQ1.1 on the JIT-DP task, evaluated on the Gerrit dataset.	43
3.5.	Comparison of encoder performance for RQ1.1 on the JIT-DP task, evaluated on the Go dataset.	44
3.6.	Comparison of encoder performance for RQ1.1 on the JIT-DP task, evaluated on the JDT dataset.	44
3.7.	Comparison of encoder performance for RQ1.2 on the CMG task, evaluated on the FIRA dataset.	45
3.8.	Performance comparison for different numbers of retrieved feature vectors (Top-k) for RQ2.1 on the APCA task, evaluated on the CACHE-Small dataset.	46
3.9.	Performance comparison for different retrieval methods of feature vectors for RQ2.2 on the APCA task, evaluated on the CACHE-Small dataset.	47
3.10.	Performance comparison for different proposed models for RQ2.3 on the APCA task, evaluated on the CACHE-Small dataset.	48
3.11.	Performance comparison for different proposed models for RQ2.3 on the JIT-DP task, evaluated on the Gerrit dataset.	48
3.12.	Performance comparison for different proposed models for RQ2.3 on the JIT-DP task, evaluated on the Go dataset.	49
3.13.	Performance comparison for different proposed models for RQ2.3 on the JIT-DP task, evaluated on the JDT dataset.	49

Appendix

A. Systematic review protocol

Jakub Zehner

This protocol describes the methodological framework for conducting a systematic review on the topic of learning code change representations via artificial intelligence code model. The review is designed according to the guidelines proposed by Kitchenham et al. [22] and SEGRESS [24], ensuring a rigorous and replicable procedure.

The principal objective of this review is to identify and classify state-of-the-art solutions in the domain of CCR. The rationale behind undertaking this systematic review includes the following objectives:

- Identify the most prevalent and effective models employed in CCR.
- Examine the types of data sets commonly used in CCR-related research.
- Explore recent advances and emerging trends in the field of CCR.

Research questions

To guide the systematic review process, the following systematic review research questions (SRRQ) have been formulated:

- *SRRQ1: What are the state-of-the-art general-purpose models for representing code and code changes?*
- *SRRQ2: What are the state-of-the-art models tailored for specific tasks, such as defect prediction or commit message generation, in the context of code and code change representation?*
- *SRRQ3: What data sets are employed in the learning of representations for code changes?*

Each research question is designed to capture different dimensions of the CCR landscape, ranging from model architectures to application-specific adaptations and data resources.

Search process

The search strategy is grounded in the best practices outlined by Kitchenham et al. [22] and further informed by SEGRESS [24]. To enhance the completeness of the review, the snowball technique, as recommended by Wohlin [23], is also used.

The initial corpus of literature consists of articles recommended by the thesis supervisor, including CCRep [3] and TransformCode [25]. Then a search query was derived from the themes of these seminal works and applied to the SCOPUS database to create a *start set*.

Following this, the snowballing process was initiated:

- **Backward snowballing** involved a systematic examination of the reference lists of selected articles.
- **Forward snowballing** was performed using Google Scholar to locate newer publications citing the selected works.

Due to time constraints inherent to the thesis schedule, the authors of the publications found were not contacted to identify possible additional sources.

Inclusion criteria

The following inclusion criteria were established to ensure relevance and alignment with review objectives.

- The study addresses the topic of CCR.
- The study qualifies as a systematic review of the literature or contributes substantially to one or more of the stated research questions.
- The study offers empirical or conceptual insights relevant to CCR.

Exclusion criteria

To maintain the quality and focus of the review, the following exclusion criteria were applied:

- The paper is not written in English.
- The paper lies outside the domain of Computer Science.
- The paper has not been peer reviewed or formally published.
- The paper is inaccessible due to paywall restrictions and cannot be obtained through institutional or open access repositories.

Data collection

For each selected study, the following metadata and descriptive information will be collected:

- Digital Object Identifier (DOI),
- Title of the publication,
- Year of publication,
- List of authors,
- Phase of the search process in which the paper was included,
- Annotated notes summarizing key insights or findings.

This data will support traceability, reproducibility, and the possibility of enriching the data set in future iterations or related studies.

Data analysis

The data collected will be organized in a structured format within a Microsoft Excel spreadsheet (.xlsx). This format facilitates efficient sorting, filtering, and annotation.

Selected papers that make significant contributions to the research questions will undergo a deeper qualitative analysis. This extended examination will be conducted within the main body of the thesis and will inform the interpretation and synthesis of the CCR research landscape.